

AI-Based Personal Exam Results Predictor Application Using Past Academic Grades

Final Project Report

Hela Udagedara Amanda Hansini

2301S0428 / BIT0523012147

Bachelor of Information Technology

Saegis Campus

Sri Lanka

August 2026

Declaration, copyright statement and statement of supervisor

I declare that this is my own work and this report does not incorporate without acknowledgement any material previously submitted for a degree or Diploma in any other University or institute of higher These guidelines were adapted from learning, and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. Also, I hereby grant to Saegis Campus the non-exclusive right to reproduce and distribute my report, in whole or in part, in print, electronic or other medium. I retain the right to use this content in whole or in part in future works.

Signature:

Date:

Name of the supervisor

Signature of the supervisor

Date:

Acknowledgement

The successful completion of this final year project, PrePal An AI-Powered Student Academic Performance Prediction and Support System, has been made possible through the guidance, encouragement, and support of many individuals and organizations, to whom I am sincerely grateful.

I wish to express my deepest and most heartfelt gratitude to my project supervisor for their unwavering academic guidance, constructive feedback, and continuous encouragement. Their expert direction in structuring the system architecture from the machine learning prediction models to the Android mobile application was invaluable in shaping the final outcome of this work. I also extend my sincere appreciation to the Faculty of Information Technology and all academic staff of the Bachelor of Information Technology (BIT) programme for the foundational knowledge and expertise imparted throughout my undergraduate studies, as well as the structured academic framework (SLQF) that informed critical features of this system.

This project was built upon the foundations of powerful open-source technologies. I gratefully acknowledge the developers and communities behind the Android SDK, Google Firebase, Python Flask, scikit-learn, and the Groq API (Llama-3.3 70B), whose tools made the rapid development and deployment of this full-stack architecture possible. I am similarly grateful to the Vite and React communities for enabling the web administration panel.

I also acknowledge the original authors of the Depression, Anxiety and Stress Scale 21 Items (DASS-21) and the National Sleep Foundation. Their validated psychometric instruments and evidence-based guidelines provided the empirical foundation necessary to bridge the gap between academic performance monitoring and student wellbeing within the PrePal warning engine.

I would like to express my profound and sincere gratitude to my family, whose boundless love, moral support, and patience have been a constant source of strength throughout my undergraduate journey. Their sacrifices and unwavering belief in my capabilities have been my greatest motivation. I also extend my heartfelt thanks to my friends and fellow BIT students for their collaborative spirit and encouragement during the challenging phases of development.

Finally, I sincerely thank all the university students who willingly participated in my survey study. Their honest responses provided the primary data used to validate the system design, identify real student pain points, and establish the evidence-based foundation of this project.

The completion of this project represents not only the culmination of my undergraduate studies but also a commitment to using technology and artificial intelligence as a force for positive academic and social impact. I dedicate this work to the betterment of student academic life.

Abstract

Academic underperformance among undergraduate students at Saegis Campus remains a significant challenge affecting module pass rates, academic progression, and degree completion. Existing academic management systems primarily focus on administrative record-keeping and retrospective performance reporting, providing limited support for predicting academic outcomes or identifying at-risk students during an active semester. This project presents PrePal, an AI-powered academic performance prediction and student support system developed for undergraduate students at Saegis Campus.

PrePal comprises a native Android mobile application, a Python-based cloud machine learning server, and a React web administration panel. Its predictive component employs a Tri-Model Multiple Linear Regression architecture consisting of Model A, the Pre-Semester Baseline Model; Model B, the Mid-Semester New Student Model; and Model C, the Comprehensive Master Model. Each model is designed for a different stage of the student academic lifecycle. The models use academic, behavioral, and wellbeing-related predictors, including class attendance, weekly self-directed study hours benchmarked against Sri Lanka Qualifications Framework standards, daily sleep duration, and perceived stress. Mid-semester models additionally incorporate continuous assessment marks. Each model produces a predicted semester GPA on the 0.00-4.00 grading scale.

The system also incorporates an Intelligent Warning Engine that evaluates student data against institutional requirements, including the minimum attendance requirement, study-hour benchmarks, and continuous assessment eligibility thresholds. Student wellbeing is assessed using the DASS-21 framework, enabling the system to generate appropriate wellbeing alerts and referral recommendations for students showing elevated psychological risk. A cloud-based backend supports secure data management, while an AI-powered chatbot provides personalized study planning, academic guidance, and wellbeing support.

A field survey conducted among Saegis Campus undergraduates indicated local consistency with the predictors identified in previous research. Higher-performing students generally demonstrated stronger attendance, adequate study habits, and lower psychological risk, whereas academically at-risk students showed lower attendance and elevated stress levels. PrePal therefore provides a proactive, data-driven, and institution-specific approach to identifying academic risk, supporting early intervention, and improving student academic decision-making across undergraduate degree programmes at Saegis Campus.

Keywords: Academic Performance Prediction, Machine Learning, Multiple Linear Regression, GPA Prediction, Student Support System, DASS-21, SLQF, Learning Analytics, Educational Data Mining, Android Application, Firebase, Saegis Campus.

Table Of Contents

1. Introduction.....	1
1.1 Overview.....	1
1.2 Background.....	1
1.3 Problem Statement.....	3
1.4 Aim and Objectives.....	4
1.4.1 Aim.....	4
1.4.2 Objectives	4
1.5 Scope and Approach	5
1.5.1 Scope.....	5
1.5.2 Approach.....	6
1.6 Assumptions.....	7
1.7 Expected Outcomes	8
1.8 Limitations	9
2. Background / Literature Review	10
2.1 Wider Context of the Project.....	10
2.2 Identified Problem	12
2.3 Stakeholders in the Problem Area.....	13
2.4 Relevant Theory and Research	14
2.4.1 Self-Regulated Learning and Student Self-Monitoring.....	14
2.4.2 Previous Achievement and Continuous Assessment.....	15
2.4.3 Attendance and Academic Performance	15
2.4.4 Study Time, Study Habits and SLQF Workload.....	15
2.4.5 Sleep and Academic Functioning.....	16
2.4.6 Wellbeing Screening and DASS-21	16
2.4.7 Educational Data Mining, Learning Analytics and Early Warning.....	17
2.4.8 AI Chatbots in Educational Support	17
2.4.9 Conceptual Synthesis and Research Gap.....	18
2.5 Constraints on the Approach.....	18

2.6 Existing Solutions and Their Limitations	19
2.7 Methods and Tools Underpinning the Solution	20
2.6.1 Literature Review, Institutional Analysis and Field Survey	20
2.6.2 Adapted Waterfall Development Method	20
2.6.3 Dataset Preparation and Tri-Model MLR	21
2.6.4 Android, Web, Cloud and API Technologies	21
2.6.5 Standards, Rules and Support Instruments	22
2.6.6 Technical Testing and Evaluation	22
2.8 Research Questions	22
3. Specification & Design (Methodology).....	23
3.1 Introduction.....	23
3.2 System Specification.....	24
3.2.1 Business Requirements	24
3.2.2 User Requirements.....	25
3.2.3 Functional Requirements	26
3.2.4 Non-Functional Requirements	29
3.3 System Design	30
3.3.1 Design Philosophy and Constraints	30
3.4 Static Architecture (High-Level System Architecture)	32
3.5 Module Design.....	33
3.6 Dynamic Behavior (System Workflows)	35
3.7 Data Flow Design	37
3.8 Data Model (High-Level)	38
3.9 Algorithms Implemented	39
3.10 Design Justification.....	41
3.11 Evolution of the Design	42
4. Implementation	46
4.1 Introduction.....	46
4.2 Frontend Implementation.....	47
4.2.1 Android Student Application.....	47
4.2.2 Student Access, Profile and Session Handling	47

4.2.3 Manual Result Entry and Academic Analytics	48
4.2.4 Lifecycle Questionnaire, Prediction and Wellbeing Interface	48
4.2.5 Calendar, Reminders and AI Chat.....	48
4.2.6 React Web Administration Panel	49
4.3 Backend Implementation	50
4.3.1 Flask Service and Request Validation.....	50
4.3.2 Grade Validation and Processing	50
4.3.3 GPA and CGPA Processing.....	50
4.3.4 Tri-Model Multiple Linear Regression	51
4.3.5 Academic Warning and DASS-21 Logic	51
4.3.6 Conversational-AI Routing.....	52
4.4 Database Implementation.....	53
4.4.1 Firestore Collections	53
4.4.2 Logical References and Lifecycle Data	54
4.4.3 Data Validation, Activity Records and Integrity	54
4.4.4 Credential and Access Data	54
4.5 Integration of System Components.....	55
4.5.1 End-to-End Student Integration	55
4.5.2 Administration-to-Student Integration	56
4.5.3 Deployment Integration	56
4.6 Testing and Debugging During Implementation.....	57
4.6.1 Frontend and Validation Testing	57
4.6.2 Google Colab Model Training and Testing.....	57
4.6.3 Backend and API Testing	57
4.6.4 Database and Integration Testing	58
4.6.5 Principal Debugging Corrections.....	58
4.6.6 Testing Boundaries.....	59
5. Results and Evaluation.....	60
5.1 Introduction.....	60
5.2 Achievement of Project Goals	60
5.2.1 Overall Goal Assessment	62

5.3 Testing Strategy.....	62
5.3.1 Test Data and Evaluation Conditions.....	63
5.3.2 Acceptance Basis	63
5.4 Functional Testing Results	63
5.4.1 Functional Findings	65
5.5 Integration Testing Results.....	65
5.5.1 End-to-End Evaluation.....	66
5.5.2 Integration Limitations.....	67
5.6 Performance Evaluation.....	67
5.6.1 Machine-Learning Evaluation	67
5.6.2 Service and Response Performance	68
5.6.3 Performance Validity	68
5.7 Usability Evaluation.....	68
5.7.1 Improvements Resulting from Evaluation	69
5.7.2 Usability Limitations	70
5.7.3 Overall Evaluation	70
5.8 Evaluation of the Spaced-Repetition Hypothesis	70
5.9 Critical Evaluation of the System	71
5.9.1 Strengths of the System	71
5.9.2 Weaknesses and Risks.....	72
5.9.3 Overall Critical Assessment.....	72
5.10 Evaluation of Methodology and Tools.....	73
5.10.1 Evaluation of the Development Methodology	73
5.10.2 Evaluation of the Selected Tools.....	73
5.10.3 Overall Methodology and Tool Assessment	74
6. Future Work	74
6.1 Introduction.....	74
6.2 Institutional Data and Model Validation	74
6.3 Prediction, Explainability and Monitoring	75
6.4 Academic Data and Result-Processing Integration.....	75
6.5 Personalized Learning and Spaced Repetition.....	76

6.6 Platform, Accessibility, Localization and Offline Support	76
6.7 Security, Privacy and Production Readiness.....	76
6.8 AI and Wellbeing Governance	77
6.9 Evaluation and Deployment Roadmap	77
6.10 Future-Work Summary	78
7. Conclusions.....	79
7.1 Introduction.....	79
7.2 Response to the Identified Problem	79
7.3 Achievement of the Aim and Objectives	79
7.4 Main Project Contributions.....	80
7.4.1 Lifecycle-Sensitive Prediction	80
7.4.2 Integrated but Distinct Forms of Support	80
7.4.3 Maintainable Distributed Architecture.....	80
7.5 Evaluation Outcome.....	81
7.6 Limitations of the Conclusion.....	81
7.7 Final Conclusion	81
8. Reflection.....	82
8.1 Reflection on Initial Assumptions.....	82
8.2 Reflection on Design Decisions.....	83
8.3 Reflection on Problem Solving Approach	84
8.4 Reflection on Personal Growth.....	85
References	86

List of Abbreviations

Abbreviation	Meaning
AB	Absent
AI	Artificial Intelligence
API	Application Programming Interface
BIT	Bachelor of Information Technology
BR	Business Requirement
CA	Continuous Assessment
CGPA	Cumulative Grade Point Average
CSV	Comma-Separated Values
DASS-21	Depression, Anxiety and Stress Scale – 21 Items
DF	Data Flow
FCM	Firebase Cloud Messaging
FR	Functional Requirement
GPA	Grade Point Average
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ID	Identifier
LMS	Learning Management System
MAE	Mean Absolute Error
MC	Medical Certificate
ML	Machine Learning
MLR	Multiple Linear Regression
MSE	Mean Squared Error
NE	Not Eligible

NFR	Non-Functional Requirement
NoSQL	Not Only Structured Query Language
OCR	Optical Character Recognition
PDF	Portable Document Format
PSS-10	Perceived Stress Scale – 10 Items
R ²	Coefficient of Determination
REST	Representational State Transfer
RMSE	Root Mean Squared Error
SDK	Software Development Kit
SHA-256	Secure Hash Algorithm – 256 Bit
SLQF	Sri Lanka Qualifications Framework
UAT	User Acceptance Testing
UI	User Interface
UR	User Requirement
WSGI	Web Server Gateway Interface
XML	Extensible Markup Language

List Of Appendices

1. Introduction

1.1 Overview

PrePal is an AI-powered student academic performance prediction and support system developed exclusively for undergraduate students enrolled across all degree programmes at Saegis Campus. The system is designed to address the absence of real-time, data-driven academic risk monitoring within the campus's existing academic management infrastructure by delivering personalized GPA predictions, intelligent risk warnings, mental health screening, and on-demand AI-assisted academic guidance all accessible from a student's Android mobile device.

The PrePal system is implemented as three integrated components:

- **Android Mobile Application:** A native Java-based Android application that serves as the primary interface for both students and administrators. Students use the app to complete academic questionnaires, receive GPA predictions, view semester performance dashboards, access an AI academic chatbot, manage personal study events, and interact with peers. Administrators use the app to manage degree programmes, student batches, semesters, modules, and student result records.
- **Python Machine Learning Server:** A Flask-based RESTful API server deployed on cloud infrastructure that hosts the Tri-Model GPA prediction engine, the intelligent academic warning system, the GPA calculation engine, and the AI chatbot interface. The server processes student behavioral and academic inputs and returns predictions, warnings, and motivational guidance.
- **React Web Administration Panel:** A TypeScript-based web dashboard built with Vite and React that provides an additional administrative interface for managing degree structures, batch enrollments, and student data, with Firebase Firestore as the shared backend database across all three components.

The system is grounded in a rigorous empirical literature review and is calibrated specifically to the academic regulations, credit structures, and examination by-laws of Saegis Campus.

1.2 Background

Academic underperformance is a documented and persistent challenge across higher education institutions globally. Research in educational data mining and learning analytics has consistently demonstrated that undergraduate Grade Point Average (GPA) is significantly influenced by a set of dynamic behavioral and psychological factors that students can actively modify, most critically: class attendance, self-directed study hours, sleep quality, and perceived stress levels.

Within the Saegis Campus context, undergraduate students across all degree programmes operate under a structured academic framework governed by the Saegis Campus Examination By-Laws, which mandate a minimum module attendance threshold of 80% (Section 6) for exam eligibility, and study-hour expectations aligned to the Sri Lanka Qualifications Framework (SLQF), which defines one credit as equivalent to 50 notional learning hours. Violations of these thresholds are penalized directly in a student's GPA, yet no existing system at Saegis Campus proactively warns students about these risks during an active semester.

The PrePal system was conceived to fill this gap. The following evidence base, gathered through a structured literature review, informed the system's design:

- Class Attendance has been identified as the strongest individual behavioral predictor of university GPA (Credé et al., 2010; $\rho = .41$ with cumulative GPA across 9,243 students), with disproportionately greater impact for academically at-risk students.
- Study Hours, when evaluated against SLQF credit-hour benchmarks, provide a measurable indicator of whether a student's independent learning effort meets institutional expectations, with a documented positive correlation with grade distributions (Zubair et al., 2024).
- Sleep Quality, assessed against the National Sleep Foundation's standard of 7–9 hours per day for adults, is linked to a statistically significant GPA depression among clinically poor sleepers (mean GPA of 2.92 ± 1.09 vs. good sleepers, $p < 0.0001$) (Maheshwari & Shaukat, 2019).
- Psychological Stress, quantified using the validated DASS-21 (Depression, Anxiety and Stress Scale 21 Items) instrument (Lovibond & Lovibond), exerts a direct and statistically significant negative impact on student academic performance through cognitive interference, working memory deficits, and maladaptive academic behaviors (Popa-Velea et al., 2021).

A primary field survey conducted among undergraduate students at Saegis Campus confirmed that these global behavioral findings mapped directly onto the local student population. Students achieving First-Class outcomes ($\text{GPA} \geq 3.70$) consistently exhibited high module attendance ($\geq 90\%$), SLQF-compliant study patterns, and low DASS-21 risk profiles, while academically at-risk students correlated strongly with attendance deterioration below the 80% institutional boundary and elevated psychological stress indicators. This local empirical validation provided the foundation for PrePal's machine learning feature selection and warning threshold calibration.

1.3 Problem Statement

Despite the availability of institutional academic records, Saegis Campus undergraduate students currently lack access to a real-time, personalized academic performance monitoring and prediction system. The following specific problems were identified as the basis for this project:

- **Absence of Real-Time GPA Prediction:** Students have no tool to predict their end-of-semester GPA based on their current behavioral patterns and academic inputs. Grade outcomes are only known after final examinations, leaving no opportunity for mid-semester intervention.
- **No Proactive Institutional Warning System:** The consequences of violating Saegis Campus Examination By-Laws such as falling below the 80% minimum attendance requirement or failing to meet SLQF study-hour standards are not communicated to students proactively during the semester. Students often discover their ineligibility for final examinations only when it is too late to remedy.
- **Inadequate Integration of Mental Health Monitoring with Academic Performance:** Student mental health, specifically depression, anxiety, and stress, has a clinically documented negative impact on academic outcomes. However, no existing system at Saegis Campus integrates mental health screening (such as the DASS-21 instrument) with academic performance monitoring to identify at-risk students and issue timely counseling referrals.
- **Lack of a Differentiated Prediction Framework Across the Student Lifecycle:** Different students have different data available at different points in the semester. A first-semester new student entering university for the first time has no prior GPA history, while a returning student mid-semester has both historical GPA and current continuous assessment (CA) marks. No existing system provides a differentiated, lifecycle-sensitive prediction model that adapts to these distinct student profiles.
- **Fragmented Academic Administration:** Academic administrators at Saegis Campus have no unified digital platform to manage degree programme structures, student batches, semester enrollments, module assignments, student result entries, and system-wide academic statistics from a single interface.
- **No Accessible AI-Powered Academic Support:** Students experiencing academic difficulties have limited access to personalized, on-demand academic guidance. Human advisors are not always available, and no AI-powered academic counseling tool exists within the campus ecosystem to provide real-time study planning, stress management advice, and module-specific academic support.

1.4 Aim and Objectives

1.4.1 Aim

This project aims to design, develop, and deploy a comprehensive AI-powered academic performance prediction and support system, **PrePal**, that enables undergraduate students across all degree programmes at Saegis Campus to monitor their academic trajectory in real time, receive personalized GPA predictions grounded in validated machine learning models, and access proactive institutional warnings and intelligent support resources to improve their academic outcomes.

1.4.2 Objectives

- To conduct a systematic literature review of empirical research in educational data mining, learning analytics, sleep science, and psychometrics to identify and validate the behavioral and psychological features with the highest predictive validity for undergraduate GPA at Saegis Campus.
- To design and train a Tri-Model Multiple Linear Regression prediction suite: Model A (Pre-Semester Baseline), Model B (Mid-Semester New Student), and Model C (Comprehensive Master Model), each calibrated to a specific stage of a Saegis Campus student's academic lifecycle, capable of predicting semester GPA on the 0.00-4.00 scale.
- To develop a multi-factor Intelligent Warning Engine that automatically monitors student inputs against Saegis Campus Examination By-Law thresholds (80% attendance, CA eligibility floors), SLQF study-hour benchmarks, National Sleep Foundation sleep guidelines, and DASS-21 mental health risk levels, and delivers structured, evidence-based alerts to students.
- To implement a dynamic, step-by-step academic questionnaire system embedded within the Android application that collects module-level attendance, study hours, sleep duration, stress level, continuous assessment marks, and DASS-21 psychological responses from students, differentiating between new first-semester students and returning students.
- To develop a dual-role native Android mobile application with a student-facing interface (GPA dashboard, performance charts, semester selector, prediction history, AI chatbot, peer chat, personal event calendar, profile management) and an administrator-facing interface (degree management, batch management, semester management, module management, student management, result entry, reporting).
- To build and deploy a Flask-based RESTful cloud API server that integrates the ML prediction models, GPA calculation engine, warning engine, and AI chatbot

powered by the Groq API (Llama-3.3-70B) with a Google Gemini fallback and exposes these capabilities to the Android client via Retrofit HTTP communication.

- To develop a React-based web administration panel for Saegis Campus administrative staff, enabling browser-based management of degree programmes, student batches, semesters, modules, and student data, with Firebase Firestore as the real-time backend database.
- To validate the system through a field survey among Saegis Campus undergraduate students and through functional testing of all system components, confirming that the ML models, warning thresholds, and application features operate correctly and produce academically meaningful outputs.

1.5 Scope and Approach

1.5.1 Scope

The scope of PrePal is limited to the implemented system and its intended Saegis Campus operating context. The principal scope boundaries are stated below.

- Institution and users: The final system is designed for Saegis Campus undergraduates across all configured degree programmes (including, but not limited to, the BIT programme) and authorized administrators who maintain the institutional data. It is highly scalable within Saegis Campus but is not configured as a solution for external institutions.
- Student platform and access: The student interface is a native Android application developed in Java and XML for Android 7.0 (API Level 24) or later. Access is controlled through a Firebase-based authentication workflow; the project does not include an iOS or web-based student application.
- Academic-result processing: The application collects academic data via a dynamic, structured in-app questionnaire. Students manually input their continuous assessment marks, previous GPA, and behavioral metrics. Automated PDF result-sheet parsing and Optical Character Recognition (OCR) are excluded from the final system due to institutional document format variability.
- Prediction function: The predictive core uses Multiple Linear Regression through three lifecycle-sensitive models. Model A provides a pre-semester baseline. Model B supports a new student at mid-semester using current mid-examination and assignment evidence. Model C provides the comprehensive pathway for a continuing student by combining previous academic performance, current continuous-assessment marks, and behavioral inputs. Predictions are returned on the Saegis Campus 0.00–4.00 GPA scale.

- Prediction variables: Depending on the selected lifecycle pathway, the predictor set uses available previous GPA, mid-examination mark, assignment mark, attendance percentage, weekly study hours, daily sleep duration, and DASS-21-derived information.
- Warning and wellbeing support: The warning engine checks the 80% attendance requirement, SLQF-derived learning-hour expectations, continuous-assessment boundaries, and DASS-21 thresholds. DASS-21 results are used strictly for screening and referral-oriented support; they are not treated as a clinical diagnosis.
- Student organization and support: The Android application includes prediction dashboards, warnings, a user profile, an academic calendar with reminders, peer-to-peer chat, and an AI academic chatbot. The chatbot provides on-demand academic and wellbeing guidance; while it can generate study plans through conversational prompts, it does not operate as a standalone, automated timetable generator.
- Administration: The React web panel provides dashboard and management functions for degree programmes, batches, semesters, modules, and student records. It supplies the maintained institutional structure used by the mobile and machine-learning workflows.
- Cloud services and deployment: Firebase Firestore is the shared real-time database. The Flask machine-learning service is deployed on Render via Unicorn, and the React administration panel is deployed for web access.

The project excludes iOS development, wearable-device integration, biometric attendance collection, postgraduate-student coverage, and direct API integration with official university management systems. Predicted GPAs and warning messages are advisory outputs; official grades and examination eligibility decisions remain under the authority of Saegis Campus.

1.5.2 Approach

The PrePal system was developed using an Agile-inspired iterative development approach, progressing through the following phases:

- I. Research and Requirements Phase: A systematic literature review was conducted to identify empirically validated GPA predictors. A primary field survey was administered to Saegis Campus students to validate local behavioral patterns and confirm system requirements.
- II. Database and Backend Design Phase: The Firebase Firestore database schema was designed to support multi-level academic hierarchies (Degrees → Batches →

Semesters → Modules → Students → Predictions). The Flask REST API server was architected to expose prediction, GPA calculation, and AI chat endpoints.

- III. Machine Learning Development Phase: The Tri-Model MLR prediction suite (Model A, B, C) was designed, trained using Python (scikit-learn, Pandas, NumPy), and evaluated using R^2 , Adjusted R^2 , MAE, MSE, and RMSE metrics. Models were serialized using joblib and integrated into the Flask server.
- IV. Android Application Development Phase: The Android mobile application was built incrementally in Java using Android Studio, implementing the splash screen, authentication, student home dashboard, quiz/questionnaire system, performance report, GPA charts (MPAndroidChart), AI chat, peer chat friends system, calendar, and profile features, alongside the full administrator module.
- V. Web Administration Panel Development Phase: The React web admin panel was developed in parallel using Vite, TypeScript, and Framer Motion, connecting to Firebase Firestore for real-time academic data management.
- VI. Integration and Testing Phase: All system components Android app, Flask ML server, and web admin panel were integrated through Firebase Firestore and Retrofit-based REST communication. The system was tested functionally across all features.
- VII. Deployment Phase: The Flask ML server was deployed to the Render cloud platform using Gunicorn as the production WSGI server.

1.6 Assumptions

The following assumptions were made during the design and development of the PrePal system:

- Student Self-Reporting Honesty: The behavioral inputs collected through the in-app questionnaire including module-level attendance percentages, weekly study hours, daily sleep duration, and stress level are assumed to be honestly and accurately reported by students. The system does not independently verify these values.
- Saegis Campus Grading Consistency: The 0.00–4.00 GPA grading scale, the SLQF credit-hour system (1 credit = 50 notional learning hours), and the Saegis Campus Examination By-Laws (including the 80% minimum attendance rule and continuous assessment eligibility thresholds) are assumed to remain consistent and unchanged throughout the operational lifespan of the system.
- Administrator Data Accuracy: Academic structure data degree programmes, batch details, semester names, module credit values, and student result records entered into the system by Saegis Campus administrators through the Android app or web panel are assumed to be accurate and complete.

- **Internet Connectivity:** The system requires an active internet connection for all core operations, including Firebase Firestore data retrieval, ML prediction API calls to the Render-deployed Flask server, and AI chatbot responses via the Groq API. Offline functionality is not implemented.
- **Android Device Availability:** All Saegis Campus students using PrePal are assumed to own or have consistent access to an Android device running Android 7.0 (API Level 24) or above.
- **DASS-21 Clinical Validity:** The DASS-21 psychometric screening instrument embedded within the questionnaire is assumed to retain its validated clinical validity (as established by Lovibond & Lovibond) when administered in a self-report digital format within the Saegis Campus student population.
- **Firebase and Cloud Service Availability:** The system depends on Google Firebase (Firestore, Storage, FCM) and the Render cloud hosting platform for core operational functions. Both services are assumed to maintain sufficient uptime and availability for normal system operations.

1.7 Expected Outcomes

The following outcomes are expected from the successful development and deployment of the PrePal system:

- I. **Real-Time GPA Predictions for Saegis Campus Students:** Students across all degree programmes at Saegis Campus will be able to obtain a predicted end-of-semester GPA at any point during the academic semester by completing the in-app questionnaire, enabling data-informed decisions about study behavior before final examinations.
- II. **Proactive Academic Risk Warning Delivery:** Students will receive timely, structured academic warnings when their inputs trigger Saegis Campus Examination By-Law violations, including attendance below 80%, CA eligibility failures, and study-hour deficits relative to SLQF standards enabling corrective action before academic penalties are incurred.
- III. **Integrated Mental Health Screening and Referral:** Students presenting with DASS-21 scores indicating moderate-to-severe depression (≥ 14), elevated anxiety (≥ 10), or high stress (≥ 19) will receive targeted mental health warnings with referrals to campus counseling services, bridging academic monitoring with student wellbeing for the first time at Saegis Campus.
- IV. **A Validated Tri-Model Machine Learning Prediction Suite:** Three trained and evaluated Multiple Linear Regression models covering pre-semester, mid-semester new student, and comprehensive master prediction scenarios will be available as a deployable ML prediction service, demonstrating measurable predictive accuracy (R^2 , MAE, RMSE) on the Saegis Campus GPA scale.

- V. A Unified Digital Academic Administration Platform: Saegis Campus administrators will have a single, unified platform accessible via both the Android app and the web administration panel to manage the complete academic structure of all degree programmes, including degrees, batches, semesters, modules, students, and result records, with real-time statistical reporting.
- VI. On-Demand AI Academic Support for Students: Students will have 24/7 access to the PrePal AI academic chatbot (powered by Llama-3.3-70B via Groq), capable of providing personalized study planning, exam preparation strategies, stress management guidance, and module-specific academic advice within the mobile application.
- VII. A Replicable System Architecture for Academic Performance Prediction: The PrePal system will serve as a documented, replicable reference architecture demonstrating how Multiple Linear Regression, DASS-21 mental health screening, SLQF academic standards, and AI-powered chatbot support can be integrated into a unified mobile academic performance system contributing a practical case study to the domain of educational data mining and learning analytics.

1.8 Limitations

While the PrePal system successfully achieves its primary objectives, it operates with certain technical, practical, and scope-related limitations. Some of these represent deviations from the original project proposal, while others reflect the constraints of a final-year academic project:

- **Reliance on Self-Reported Data:** The system relies entirely on students accurately and honestly self-reporting their module-level attendance, weekly study hours, and daily sleep duration via the in-app questionnaire. It does not integrate with biometric attendance scanners, university Learning Management Systems (LMS), or wearable devices to automatically verify these behavioral inputs.
- **Absence of PDF Result Extraction (Deviation):** The initial project proposal stated the objective to allow students to upload academic results in PDF format. In the final implementation, automated PDF parsing and optical character recognition (OCR) were excluded due to the high variability and unstructured nature of institutional result PDFs. Consequently, the system relies exclusively on the manual entry of academic grades.
- **Study Plan Generation via AI Chatbot (Deviation):** The original proposal aimed to generate personalized study plans directly from predicted outcomes. While successfully implemented, this feature was integrated directly into the conversational AI Chatbot interface rather than functioning as a standalone automated dashboard generator.

- **Platform Restriction:** The student-facing mobile application was developed exclusively for the Android platform (minimum SDK 24 / Android 7.0). There is no iOS counterpart, which restricts access for students using Apple devices.
- **No Direct Institutional API Integration:** The system does not directly pull data from Saegis Campus's official student management system. All academic structures (modules, batches) and student result records must be manually configured and managed by the administrator through the PrePal application or web panel.
- **Offline Unavailability:** The core predictive and support features, including the Tri-Model prediction engine and the Groq/Gemini-powered AI chatbot, require real-time network communication with the Firebase backend and Render-hosted Flask server. The application does not offer offline prediction capabilities.
- **Screening vs. Clinical Diagnosis:** The integration of the DASS-21 instrument provides psychometric screening and generates referral-oriented warnings. However, the system is strictly an academic support tool; it does not, and cannot, provide formal clinical psychiatric diagnoses.
- **Prediction Unavailability for Brand-New Students:** The system requires at least some academic baseline to function. First-year, first-semester students who have just commenced lectures cannot receive a GPA prediction until they have completed and received marks for their mid-semester examinations and initial assignments (the minimum requirement for Model B).
- **Handling of Special Grades (MC, AB):** The ML prediction models require numeric performance baselines to function accurately. Students who have unresolved special administrative grades from previous semesters such as MC (Medical Certificate) or AB (Absent) cannot obtain a valid GPA prediction until those modules are completed and converted into standard academic grades.

2. Background / Literature Review

2.1 Wider Context of the Project

Higher education institutions generate substantial amounts of academic information through admissions, module enrolment, attendance, continuous assessment and final examinations. However, the existence of data does not automatically provide students with timely or understandable support. Conventional student information systems are primarily designed to record enrolment and publish confirmed results, whereas learning analytics focuses on transforming educational data into evidence that can support awareness, prediction and intervention. The wider context of PrePal is therefore the movement from retrospective academic reporting toward student-centered, preventive academic support.

Educational data mining and learning analytics research demonstrates that patterns contained in academic records can be used to estimate performance and identify students

who may require assistance. Shahiri, Husain and Rashid (2015) found that previous academic performance and cumulative grade information are among the most frequently used attributes in student-performance prediction research. Yağcı (2022) similarly demonstrated that mid-semester achievement data can support early estimation of final performance while time remains for corrective action. These findings support the use of previous GPA and current continuous-assessment marks in PrePal, while also showing that a prediction is only meaningful when the required data are actually available.(Shahiri, Husain and Rashid, 2015)(Yağcı, 2022)

The need for early support is also consistent with learning-analytics early-warning systems. Purdue University's Course Signals combined academic performance, effort, and engagement data to provide students with frequent performance notifications and routes to support (Arnold & Pistilli, 2012). The broader principle is that a useful system should not simply label a student as being at risk; it should provide an understandable signal early enough to encourage reflection and action. PrePal applies this principle at the individual-student level by combining a predicted semester GPA with explicit warnings and explanatory support.(Dawson, 2012)

Academic performance is multidimensional. Previous grades and continuous-assessment marks provide direct evidence of achievement, but attendance, study effort, sleep and wellbeing can influence the conditions under which learning occurs. This does not mean that any single behavior determines GPA. Instead, the literature supports treating these factors as related indicators whose combined contribution may be examined through a transparent prediction model and complemented by rule-based checks where an institutional policy establishes a clear boundary.

The Sri Lankan higher-education context adds an important workload dimension. The Sri Lanka Qualifications Framework states that one credit for a taught, laboratory or field-studies course represents 50 notional learning hours and that workload should reflect the total learning activities required to achieve the intended outcomes (University Grants Commission, 2015). Notional learning includes more than independent study alone; it also encompasses contact learning, preparation, assessment and related academic activity. PrePal uses the framework to derive an expected workload reference while clearly treating self-reported study hours as an indicator rather than a complete measurement of all learning activity.(Sri Lanka Qualifications Framework (SLQF))

The project also recognizes that academic risk and student wellbeing should not be treated as unrelated areas. The DASS-21 offers a short self-report structure for measuring the related emotional states represented by its depression, anxiety and stress subscales((Lovibond & Lovibond, 1995; Antony et al., 1998)). In PrePal, DASS-21 is used for awareness and referral-oriented messages, not diagnosis. This boundary is important

because an educational application can make students aware of concerning questionnaire results but cannot replace qualified professional assessment.

Mobile delivery is appropriate to the intended student-centered role because students require direct access to their own analytics, reminders and support rather than depending entirely on an administrator or lecturer to interpret institutional records. At the same time, institution-wide operation requires a controlled source of programme, batch, semester, module, student and policy data. PrePal therefore separates the student experience into an Android application and the administrative experience into a React web panel, while a Flask prediction service and Cloud Firestore connect the required data and services.

PrePal is designed for undergraduates across all degree programmes at Saegis Campus. Institution-wide design does not remove the need for programme-specific configuration: module structures, credits, grading rules and result-sheet layouts can differ. The project consequently combines a reusable architecture with administrator-maintained academic structures. The implemented prototype demonstrates the integrated workflow, while its limitations acknowledge that comprehensive validation across every programme, result format and student population remains future work.

2.2 Identified Problem

The central problem is the absence of one student-facing system at Saegis Campus that can convert available academic and behavioral information into a timely, lifecycle-appropriate GPA estimate and then connect that estimate to academic rules, wellbeing awareness, historical analytics and practical support. The problem can be divided into the following related gaps:

- Retrospective rather than predictive information. Students normally receive confirmed academic results after the relevant assessment period. Such records are essential, but they do not estimate likely semester performance while the student can still adjust attendance, study effort, or assessment preparation.
- Fragmented interpretation of academic records. A result sheet does not automatically explain semester GPA, CGPA, repeated weak modules or the direction of performance across semesters. Students may therefore view grades without understanding their cumulative effect.
- Different information at different academic stages. A brand-new student has no previous university GPA, while a continuing student may possess both a previous GPA and current continuous-assessment marks. A single fixed questionnaire would either demand unavailable information or ignore evidence that is already available.
- Separation of statistical prediction and institutional rules. A regression estimate alone cannot determine whether attendance is below a mandatory threshold, study effort is below a configured workload reference or a continuous-assessment

condition creates a specific risk. These conditions require explicit rule-based interpretation.

- Limited connection between academic support and wellbeing awareness. Academic monitoring tools often exclude structured wellbeing screening, while standalone screening tools do not explain academic performance. An integrated but carefully bounded approach is required.
- Disconnected student and administrator workflows. Institution-wide prediction requires current programme, batch, semester, module, policy and student information. Direct database editing is impractical, while a student application should not expose administrative controls.
- Lack of immediate explanatory support. Charts and warnings identify patterns but may not answer a student's follow-up questions. Students benefit from conversational explanations and general guidance, provided that AI-generated responses remain advisory.

The project field survey among Saegis Campus undergraduates provided local support for these gaps. Reported patterns linked stronger academic standing with more favorable attendance, study, and wellbeing profiles, while academically at-risk patterns were associated with less favorable indicators. The survey was used as descriptive local evidence for feature selection and not as proof of causation or as the direct training dataset for the final prediction models.

2.3 Stakeholders in the Problem Area

The problem affects several stakeholder groups, although the completed system provides direct interfaces only to students and authorized administrators. The remaining stakeholders are indirect beneficiaries or governance participants rather than separate application roles.

- Undergraduate students. Students across Saegis Campus degree programmes are the primary beneficiaries. They manually enter their academic results, review GPA and CGPA, inspect performance charts, complete the appropriate prediction questionnaire, receive warnings, use calendar and reminder functions and access the AI academic chatbot. Their main interests are understandable feedback, privacy, accuracy, timely access and clear advisory boundaries.
- Authorized academic administrators. Administrators maintain degree programmes, batches, semesters, modules, student records and policy information through the web panel. They also perform CSV operations and review activity information. Their responsibility is data quality because incorrect academic structures or dates can affect student eligibility, model selection and displayed information.
- Lecturers and academic advisers. Lecturers and advisers are indirect stakeholders because students may use PrePal outputs to identify questions, seek clarification or

request academic support. The system does not replace professional academic judgement and does not provide a dedicated lecturer interface in the completed scope.

- Student-support and counselling personnel. Campus support personnel are indirect stakeholders when the application presents referral-oriented wellbeing messages. PrePal does not share a clinical diagnosis or replace a professional service; its role is limited to awareness and directing a student toward appropriate support.
- Institutional management and programme coordinators. Management and coordinators have an interest in consistent programme configuration, compliance with academic policies and the responsible use of predictive technology. Wider institutional deployment would require governance, privacy, security and validation decisions beyond the prototype.
- System developer and future maintainers. The developer and any future technical team are responsible for maintaining the Android client, web panel, Firestore structures, prediction API, security controls and external-service integrations as programmes and policies change, utilizing manual data entry where systems are not automated.

Stakeholder interests are not identical. Students need accessible and supportive information, administrators need controlled and accurate data management, and the institution requires reliable and ethically governed operation. PrePal addresses these differences through role separation, advisory wording, and a clear distinction between predictive support, official academic decisions, and professional wellbeing assessment.

2.4 Relevant Theory and Research

2.4.1 Self-Regulated Learning and Student Self-Monitoring

Self-regulated learning theory provides a useful educational foundation for PrePal. Zimmerman (2002) describes self-regulation as a cyclical process involving forethought, performance and self-reflection. Students set goals, apply strategies, monitor progress and then evaluate outcomes. A system that only publishes final grades supports reflection after the event but offers limited assistance during forethought and performance. PrePal extends the information available during these earlier stages through GPA trends, weak-module identification, prediction, warnings, reminders and conversational guidance. (Zimmerman, 2002)

The theory also explains why interpretability matters. A student is more likely to act on feedback when the relationship between the result and the relevant behavior is understandable. For this reason, PrePal does not present the predicted GPA as an unexplained score. The lifecycle model identifies the information used at that stage, while the warning engine communicates specific attendance, workload, continuous-assessment

or wellbeing conditions. The system supports reflection and planning but leaves academic decisions with the student and institution.

2.4.2 Previous Achievement and Continuous Assessment

Previous achievement is consistently important in student-performance prediction research. The review by Shahiri et al. (2015) identified cumulative GPA and prior grades as common predictive attributes. Previous GPA summarizes sustained performance across completed modules and therefore provides a reasonable baseline for continuing students. It is not available for a new university entrant, which motivates the need for a separate lifecycle pathway rather than substituting an invented or default value.(Shahiri, Husain and Rashid, 2015)

Current continuous-assessment marks provide more immediate evidence of how a student is performing in the active semester. Yağcı (2022) used midterm information to predict final achievement and emphasized the value of making an estimate while time remains for corrective action. Suárez, Llanos, and Bucheli (2023) similarly examined regression-based final-grade prediction using the portion of course grades already available. These studies support the inclusion of mid-examination and assignment marks in PrePal's applicable mid-semester models, while also justifying the rule that such fields should not be requested before the assessments exist.(Yağcı, 2022)

2.4.3 Attendance and Academic Performance

Attendance represents direct participation in scheduled learning activities and is one of the strongest research-supported academic indicators used in PrePal. Credé, Roch and Kieszczynka (2010) synthesized 69 samples involving 21,195 students for class grades and 33 samples involving 9,243 students for GPA. Their meta-analysis reported relationships of approximately 0.44 with class grades and 0.41 with GPA. This evidence supports attendance as a predictor but does not imply that attendance alone determines performance. Attendance also has a different role when institutional rules establish a minimum eligibility level. A regression coefficient estimates an average statistical relationship; it cannot enforce or explain a policy boundary. The implemented warning engine therefore checks the configured 80% requirement independently and presents a direct warning when the student's reported value falls below it. This hybrid design separates statistical estimation from deterministic institutional interpretation.(Credé, Roch and Kieszczynka, 2010)

2.4.4 Study Time, Study Habits and SLQF Workload

Research on study time shows that the relationship with academic performance is influenced by the quality of study habits and the context in which time is used. Nonis and Hudson (2010) reported that study time should not be interpreted independently of study habits and other student characteristics. Consequently, PrePal treats weekly study hours as one indicator within a wider profile rather than assuming that more hours always produce a higher GPA.(Nonis and Hudson, 2010)

The SLQF provides an institutionally relevant basis for interpreting workload. It defines one credit for taught, laboratory and field-studies courses as 50 notional learning hours and requires workload to reflect the learning activities needed to achieve module outcomes (Sri Lanka Qualifications Framework (SLQF)). PrePal derives a weekly reference from programme credits and semester duration. Because notional learning includes contact time, independent study, assignment preparation and assessment, the warning is framed as a workload indicator and not as a precise measurement of productive study.

2.4.5 Sleep and Academic Functioning

Sleep supports attention, memory and the learning processes required for academic work. Maheshwari and Shaukat (2019) found that medical students classified as poor sleepers had a significantly lower mean GPA than good sleepers in their sample. Taylor et al. (2013) also examined sleep as a predictor of college academic performance alongside psychological, demographic and educational factors. These studies support the inclusion of daily sleep duration as part of a multidimensional profile. (Maheshwari and Shaukat, 2019) (Taylor *et al.*, 2013)

The evidence must be interpreted carefully. The cited studies identify associations and do not establish that a particular number of sleep hours directly causes a specific GPA. Sleep may also interact with workload, stress, health and personal circumstances. PrePal therefore uses sleep as one model input and a source of general wellbeing guidance rather than as an eligibility rule or deterministic judgement.

2.4.6 Wellbeing Screening and DASS-21

Wellbeing can influence concentration, motivation, sleep and engagement, but it should not be reduced to an academic variable. The DASS framework was developed to distinguish three related negative emotional-state dimensions, and the 21-item form provides seven items for each subscale (Lovibond & Lovibond, 1995; Antony et al., 1998). The instrument is self-reported and intended for measuring symptom severity within its defined scales, not for providing a clinical diagnosis. (Antony *et al.*, 1998)

Research has begun to combine psychological evaluation with academic prediction. Zhang et al. (2024) demonstrated that psychological and course data can be integrated in a machine-learning GPA-prediction framework. For PrePal, the more important design implication is responsible integration: DASS-21 is scored separately, used for awareness and referral-oriented messages and clearly labelled as non-diagnostic. The system avoids presenting a questionnaire category as a confirmed condition and does not replace campus counselling or qualified professional assessment. (Shahiri, Husain and Rashid, 2015; Zhang *et al.*, 2024)

2.4.7 Educational Data Mining, Learning Analytics and Early Warning

Educational data mining applies computational methods to discover patterns in educational data, while learning analytics emphasizes the interpretation and use of those patterns to support learning and decision-making. Shahiri et al. (2015) reviewed common prediction attributes and algorithms, and Yağcı (2022) showed how mid-semester data can support early performance classification. These studies demonstrate the technical feasibility of prediction but also show that model results depend strongly on the population, variables and data quality.(Shahiri, Husain and Rashid, 2015)(Yağcı, 2022)

Early-warning research adds a communication and intervention perspective. Course Signals used existing academic and engagement data to send frequent performance notifications and direct students toward support(Dawson, 2012). Akçapınar, Altun and Aşkar (2019) likewise investigated learning-analytics early-warning methods for identifying students at risk. PrePal adopts the early-support principle but differs in being a student-facing application that combines self-monitoring, prediction, explicit institutional rules, wellbeing awareness and conversational assistance.(Akçapınar, Altun and Aşkar, 2019)

Multiple Linear Regression was selected because the project predicts a continuous GPA value and requires a model whose coefficients can be explained. The model can be represented as $y = \beta_0 + \beta_1x_1 + \dots + \beta_nx_n$, where y is the estimated semester GPA and each β expresses the fitted contribution of an input while other inputs are held constant. The approach is transparent and computationally efficient, but it assumes linear relationships and does not capture every possible interaction or nonlinear effect. This limitation is particularly important because the final prototype uses synthetic training data.

2.4.8 AI Chatbots in Educational Support

Educational chatbots can provide immediate access to explanations, guidance and information. Wollny et al. (2021) reviewed 74 relevant publications and identified roles including learning support, mentoring and personalized interaction. Labadze, Grigolia and Machaidze (2023) similarly reported benefits such as quick assistance and access to information while identifying concerns involving accuracy, privacy, evaluation and responsible use.(Wollny *et al.*, 2021)

These findings support the inclusion of the PrePal chatbot as an explanatory and guidance layer rather than an autonomous academic decision maker. The implemented chatbot can discuss performance information, study approaches, examination preparation and general non-clinical wellbeing guidance. However, responses depend on the information provided and the external language model. The system therefore retains advisory wording and does not claim that chatbot output is equivalent to lecturer, adviser or professional guidance.

2.4.9 Conceptual Synthesis and Research Gap

The literature supports each major element of PrePal but also reveals fragmentation. Academic prediction research often concentrates on algorithm performance; early-warning systems are frequently institution-controlled; GPA calculators are retrospective; wellbeing tools are separate from academic analytics; and educational chatbots may not have access to institution-specific rules or the student's academic lifecycle. The original proposal identified the need to bring these capabilities together in a mobile student-support environment.

The final research gap is therefore not the absence of every individual technology. It is the absence, within the Saegis Campus context, of a unified student-centred system that combines supported result processing, GPA and CGPA analytics, lifecycle-sensitive semester-GPA prediction, institution-specific rule warnings, DASS-21 awareness, calendar support, administrator-maintained academic structures and conversational academic guidance. PrePal addresses this integration gap while acknowledging that institution-wide empirical validation, automated official-system integration and several originally proposed functions remain outside the completed prototype.

2.5 Constraints on the Approach

The solution approach was shaped by technical, data, ethical and project constraints. These constraints influenced both the selected methods and the boundaries placed around the system's outputs.

- Data availability. A sufficiently large official longitudinal Saegis Campus dataset was not available for model development. A 1,000-record synthetic dataset was therefore used, limiting claims to prototype-level technical evaluation.
- Self-reported predictors. Attendance, study hours, sleep, current marks, and questionnaire responses depend on student accuracy. The project did not integrate biometric attendance, wearable devices or LMS activity data.
- Model simplicity and interpretability. Multiple Linear Regression supports transparent coefficient interpretation but assumes linear relationships and may omit nonlinear interactions.
- Academic lifecycle. The applicable prediction variables change with semester status and data availability. Brand-new students cannot receive a meaningful estimate before the minimum current assessment evidence for Model B exists.
- Institution-wide configuration. The architecture is designed for all undergraduate programmes, but each programme requires accurate module, credit, grading, semester and result-layout configuration.
- Absence of PDF extraction. Automated PDF parsing and optical character recognition (OCR) were excluded due to the high variability and unstructured nature of institutional result PDFs. Consequently, the system relies exclusively on the manual entry of academic grades.

- Infrastructure dependence. The Android client requires network access for Firestore data, the Render-hosted prediction API, and external AI services. Offline prediction was not implemented.
- Ethical and professional boundaries. Predicted GPA is advisory, DASS-21 is non-diagnostic, and chatbot responses are general guidance. Official academic and professional decisions remain outside the system.
- Final-year project resources. The system was completed by one student within a fixed academic schedule. Large-scale usability testing, longitudinal effectiveness evaluation, iOS development and official-system APIs were not completed. The proposal's prediction-confidence indicator and past-paper access functions were also not implemented, while personalized planning was delivered through the conversational AI chatbot instead of as a separate automated study-plan dashboard.

2.6 Existing Solutions and Their Limitations

Existing solutions address parts of the problem, but no reviewed category fully matches the completed integration required for PrePal. Table 2.1 summarizes the comparison at solution-category level rather than claiming that every product has identical functions.

Solution category	Typical capability	Limitation relative to the PrePal problem
Student information systems and institutional portals	Record enrolment, modules and confirmed results.	Primarily administrative and retrospective; students may not receive personal prediction, behavioral interpretation, wellbeing awareness or conversational support.
LMS analytics and early-warning platforms	Use grades, activity and engagement data to identify risk.	Often institution-controlled, dependent on LMS data and course-level implementation, and not designed as an independent student self-monitoring application.
GPA calculators and spreadsheets	Calculate GPA or CGPA from entered grades.	Explain completed performance but usually do not apply lifecycle-sensitive machine learning, institutional warning rules or DASS-21 screening.
Standalone wellbeing screeners	Provide questionnaire scoring and general information.	Separate from academic records and prediction; they cannot explain how academic and behavioral indicators

		appear together and must remain non-diagnostic.
Generic AI chatbots	Answer questions and generate general study guidance.	May not know the student's verified academic structure, current semester stage or institution-specific policies; responses can be inaccurate and require human judgement.
Calendar and reminder applications	Store events and issue reminders.	Support organization but do not connect dates with GPA analytics, risk prediction, academic rules or programme administration.

Table 2.1: Existing solution categories and the remaining integration gap

PrePal's contribution is the coordinated workflow across these categories. The Android application gives students direct access to their own results, analytics, predictions, warnings, calendar and chatbot. The web administration panel provides controlled maintenance of institutional structures, while the Flask service performs lifecycle model selection and rule evaluation. This integration is the principal difference; it does not imply that PrePal replaces the official student information system, LMS, lecturer, counsellor, or institutional decision-making process.

2.7 Methods and Tools Underpinning the Solution

The implemented solution is underpinned by a combination of research methods, statistical modelling, mobile and web engineering, cloud data management, document processing and rule-based support.

2.6.1 Literature Review, Institutional Analysis and Field Survey

The literature review was used to identify evidence for the selected predictors and support mechanisms. Academic-performance prediction studies informed the use of previous GPA and continuous-assessment marks; attendance, study-time and sleep research informed the behavioural inputs; DASS literature established the screening boundary; and learning-analytics and chatbot studies informed the warning and support design. Saegis Campus academic rules and the SLQF were analysed to identify institutionally meaningful conditions. A descriptive field survey among Saegis Campus undergraduates supplied local supporting evidence and was not treated as causal proof or model-training data.

2.6.2 Adapted Waterfall Development Method

The final project is documented using an adapted Waterfall method comprising requirements and scope confirmation, evidence analysis, system and data design, dataset and model development, application development, integration and deployment, and

technical testing and refinement. The method provided a sequential structure suitable for an individual project while allowing corrections identified during development and testing to be applied before final delivery.

2.6.3 Dataset Preparation and Tri-Model MLR

A 1,000-record synthetic dataset was generated to represent plausible relationships among previous GPA, mid-examination marks, assignment marks, attendance, study hours, sleep and wellbeing-related information. Multiple Linear Regression was implemented using scikit-learn. Model A supports the pre-semester baseline, Model B supports a new student after current continuous-assessment evidence becomes available, and Model C supports the comprehensive continuing-student case. The service selects the model according to lifecycle and data availability rather than requiring one fixed input set.

The models were evaluated with Mean Absolute Error, Mean Squared Error and the coefficient of determination (R-squared). MAE expresses average absolute prediction error in GPA units, MSE gives greater weight to larger errors, and R-squared indicates the proportion of outcome variance explained relative to a mean baseline. These metrics provide technical evidence about the synthetic test data; they do not constitute institutional validation on real student histories.

2.6.4 Android, Web, Cloud and API Technologies

- Android student application. Java and XML were used to implement account access, profile functions, manual result entry, GPA and CGPA calculations, charts, the prediction questionnaire, DASS-21 processing, warnings, calendar, reminders and AI-chatbot access.
- Machine-learning service. Python, Flask and scikit-learn implement input validation, lifecycle model selection, GPA prediction and warning logic. Unicorn serves the API, which is deployed through Render.
- Web administration panel. React, Vite and TypeScript implement the administrator dashboard and management functions for programmes, batches, semesters, modules, students, policies, CSV operations and activity records.
- Cloud data layer. Cloud Firestore stores the structured records shared by the student and administration workflows. The prototype uses a custom Firestore-based account-verification and session approach rather than claiming Firebase Authentication.
- Manual grade entry. Students manually enter their academic grades into the application, which are then validated against the configured student and module structure before GPA processing.
- Conversational AI. The Android chatbot uses the implemented Groq service with Google Gemini fallback to provide general academic explanations, study guidance and non-clinical wellbeing support. Its responses remain advisory and network-dependent.

2.6.5 Standards, Rules and Support Instruments

The warning engine combines configured Saegis Campus academic rules with SLQF workload interpretation. The 80% attendance condition is checked directly rather than inferred from the regression model. Credit-based workload references are derived from the SLQF definition of 50 notional learning hours per taught-course credit and the configured semester duration. Continuous-assessment warnings are applied only when the relevant marks and policy conditions exist.

DASS-21 scoring is performed separately from the official academic decision process. The application uses the resulting categories to select awareness and referral-oriented messages, explicitly states that the result is non-diagnostic and avoids presenting the questionnaire as a professional assessment. This separation allows wellbeing awareness to be included without exceeding the role of an academic support application.

2.6.6 Technical Testing and Evaluation

Testing covered principal Android and web workflows, account and role separation, academic-data management, manual grade entry, grade flags, GPA and CGPA calculations, chart presentation, lifecycle model selection, API responses, warning conditions, DASS-21 messages, calendar behavior, chatbot service fallback and Firestore synchronization. Model evaluation used the three-regression metrics described above. The project did not complete a large-sample institution-wide usability study or a longitudinal study of changes in final academic outcomes.

2.8 Research Questions

The literature, identified problem, institutional context, and implemented scope lead to the following research questions:

- I. To what extent can a Tri-Model Multiple Linear Regression architecture estimate semester GPA using the academic, continuous-assessment, behavioral and wellbeing-related information available at different stages of an undergraduate student's academic lifecycle?
- II. How can lifecycle-sensitive model selection support brand-new and continuing students without requesting previous GPA or continuous-assessment values before those data exist?
- III. How can rule-based interpretation of attendance, SLQF-derived workload expectations and continuous-assessment conditions complement a statistical GPA prediction?
- IV. How can DASS-21 wellbeing screening be integrated into an academic-support application while preserving non-diagnostic and referral-oriented boundaries?
- V. How can manual grade entry, GPA and CGPA calculations, weak-module visualization and semester trends help present academic performance in an understandable student-facing form?

- VI. How can an Android student application, a web administration panel, Cloud Firestore and a Flask prediction service be integrated to support undergraduates across Saegis Campus degree programmes while retaining role separation and programme-specific configuration?
- VII. How can an AI academic chatbot explain analytics and provide general guidance without being presented as an autonomous academic adviser or professional wellbeing service?

These questions are intentionally aligned with what the completed prototype can demonstrate. They focus on technical feasibility, integration, lifecycle handling and responsible support. Questions requiring evidence of institution-wide academic improvement, clinical effectiveness or long-term behavioral change are excluded because the project did not conduct the large-scale longitudinal studies necessary to answer them.

3. Specification & Design (Methodology)

3.1 Introduction

This chapter translates the problem definition and research foundation into the specification and technical design of the completed PrePal prototype. It identifies the business, user, functional, and non-functional requirements; explains the high-level architecture and module boundaries; documents the principal workflows, data flows, and logical data model; and presents the algorithms implemented for result processing, academic analytics, GPA prediction, warning generation, and student support. Crucially, this chapter describes the final system as implemented and does not treat incomplete proposal features as delivered functionality.

PrePal was developed using an iterative, Agile-inspired methodology. Rather than strictly adhering to formal frameworks like Scrum (e.g., fixed sprint durations or daily stand-ups), the project adopted core Agile principles adapted for this project's context. Requirements were continuously refined as each functional increment was designed, implemented, demonstrated, and evaluated through a feedback-driven process.

The completed solution comprises a native Android student application written in Java and XML, a React/Vite/TypeScript web administration panel, Cloud Firestore as the shared structured-data layer, and a Python Flask service deployed through Render for document processing, lifecycle-sensitive Multiple Linear Regression prediction, and warning logic. Furthermore, the Android application provides access to an academic chatbot, utilizing Groq as the primary service with a Gemini fallback mechanism.

Methodology definition: Agile refers to the actual feedback-driven and incremental process used in the project. The report does not claim that a complete formal Scrum framework, fixed sprint duration, daily stand-ups, or a multi-person Scrum team was used.

3.2 System Specification

The system specification defines what PrePal was required to achieve and the boundaries within which the final prototype operates. Requirements are grouped by institutional need, user need, functional behavior and quality attributes. The identifiers in this chapter provide traceability between the stated requirement, the implemented component and the later testing and evaluation discussion.

3.2.1 Business Requirements

ID	Business requirement	Implemented response
BR-01	Provide undergraduates across Saegis Campus degree programmes with a single student-facing environment for monitoring academic performance.	Android application combining result analysis, prediction, warnings, calendar functions and academic support.
BR-02	Provide a semester-GPA estimate while sufficient academic evidence still exists for a student to review study behavior and preparation.	Tri-Model MLR service selects an appropriate lifecycle pathway and returns an advisory prediction.
BR-03	Convert student academic results into understandable semester and cumulative performance information.	Manual grade entry with grade interpretation, GPA/CGPA calculations, weak-module chart and semester trend.
BR-04	Relate prediction inputs to applicable academic rules rather than presenting an unexplained statistical value.	Deterministic warning checks for the configured 80% attendance rule, continuous-assessment conditions and SLQF-derived workload expectations.
BR-05	Include responsible wellbeing awareness without presenting an academic application as a clinical service.	DASS-21 screening categories are used only for non-diagnostic awareness and referral-oriented messages.
BR-06	Allow authorized administrators to maintain the academic structures required by students and the prediction workflow.	React web panel manages programmes, batches, semesters, modules, students, policies, CSV operations and activity records.

BR-07	Restrict access to provisioned campus users and separate student and administrator privileges.	Student Android and administrator web interfaces use separate role-aware access paths and Firestore-backed account/session records.
BR-08	Provide timely, on-demand explanatory academic support in addition to charts and warnings.	The Android chatbot supplies general academic, study and non-clinical wellbeing guidance through Groq with Gemini fallback.
BR-09	Maintain an architecture that can represent multiple undergraduate programmes while permitting programme-specific configuration.	Programme, batch, semester, module, grading and policy data are administrator-maintained rather than hard-coded into one fixed course structure.

Table 3.1: Business requirements and their implemented responses

3.2.2 User Requirements

The completed prototype has two direct user classes: undergraduate students and authorized academic administrators. Lecturers, advisers, counsellors and management are stakeholders, but the final system does not provide separate interfaces for those groups.

ID	User	Requirement
UR-S01	Student	Sign in using a provisioned student identity and password and access only the student interface.
UR-S02	Student	View and maintain a personal profile and the academic context associated with the student's configured batch.
UR-S03	Student	Manually input academic results for a completed semester and receive a clear success or validation message when grades are saved.
UR-S04	Student	View semester GPA, CGPA, weak-module information and the direction of performance across available semesters.
UR-S05	Student	Complete only the prediction fields relevant to the current semester stage and available academic evidence.
UR-S06	Student	Receive an advisory predicted GPA together with reasons and academic-rule warnings where conditions are triggered.
UR-S07	Student	Complete the DASS-21 screening component and receive clearly labelled, non-diagnostic awareness or referral messages.
UR-S08	Student	Create and review calendar events and reminders that support academic organization.

UR-S09	Student	Ask the AI chatbot for explanations, study guidance and general non-clinical support.
UR-S10	Student	Receive understandable error messages when network services or required prediction data are unavailable.

Table 3.2: Undergraduate student requirements

3.2.3 Functional Requirements

Functional requirements define the observable operations performed by the three principal application components. They are grouped below to avoid presenting unimplemented proposal ideas as system functions.

ID	Functional requirement	Implemented component
FR-ACC-01	Verify a student's provisioned identifier and password before granting access to the Android application.	Android account/session module and Firestore account records
FR-ACC-02	Verify an authorized administrator before granting access to the web administration panel.	React administration access module and Firestore administrator records
FR-ACC-03	Maintain student and administrator role separation and redirect each authenticated role to its own interface.	Separate Android and web clients
FR-ACC-04	Maintain a local session state for the authenticated Android user and clear it during sign-out.	SharedPreferences-based student session
FR-ACC-05	Store password representations as hashes rather than readable plain-text values.	Custom Firestore-backed credential logic

Table 3.4: Access and account functional requirements

ID	Functional requirement	Implemented component
----	------------------------	-----------------------

FR-RES-01	Accept manual entry of academic grades through the Android result-entry workflow.	Android manual entry module
FR-RES-02	Ensure manually entered grades are correctly mapped to the respective modules for the selected semester.	Grade mapping logic
FR-RES-03	Validate the student's entered grades against the configured module structure and normalize module-grade values.	Grade validation rules
FR-RES-04	Flag special values such as AB, MC and NE instead of treating them as ordinary numeric grades.	Grade interpretation logic
FR-RES-05	Calculate credit-weighted semester GPA and cumulative GPA from processed results.	Academic analytics module
FR-RES-06	Display weak modules in a bar-chart view and available semester GPA values in a trend view.	Android visualization module

Table 3.5: Result-processing and analytics functional requirements

ID	Functional requirement	Implemented component
FR-PRD-01	Retrieve batch, semester and availability information needed to determine the current prediction pathway.	Android/Firestore context logic
FR-PRD-02	Request previous GPA only when it exists and current mid-semester/assignment marks only when those assessments are available.	Lifecycle-sensitive questionnaire
FR-PRD-03	Validate prediction inputs and select Model A, B or C according to the available academic evidence.	Flask lifecycle router
FR-PRD-04	Return a semester-GPA estimate on the Saegis GPA scale using the selected Multiple Linear Regression model.	Scikit-learn Tri-Model service
FR-PRD-05	Evaluate attendance, configured continuous-assessment conditions and SLQF-derived workload expectations independently of the regression estimate.	Rule-based warning engine

FR-PRD-06	Process DASS-21 responses separately and return non-diagnostic awareness or referral-oriented messages.	DASS-21 scoring and warning logic
FR-PRD-07	Explain when a prediction cannot be produced because the minimum lifecycle evidence is unavailable.	Input-validation response

Table 3.6: Prediction and warning functional requirements

ID	Functional requirement	Implemented component
FR-SUP-01	Allow a student to add, view, and manage academic calendar events and reminders.	Android calendar/reminder module
FR-SUP-02	Send an academic prompt to the configured AI service and present the returned response in the Android application.	AI chat module
FR-SUP-03	Use Groq as the primary AI service and attempt Gemini fallback when the primary route is unavailable.	Chatbot service-routing logic
FR-SUP-04	Present chatbot output as general guidance and not as an official academic or clinical decision.	Prompt and interface safeguards

Table 3.7: Student-support functional requirements

ID	Functional requirement	Implemented component
FR-ADM-01	Create, view, update and remove or deactivate degree-programme, batch, semester, module and student records.	React administration modules
FR-ADM-02	Maintain semester start/end dates and active, completed or inactive status values used by student lifecycle logic.	Semester administration module
FR-ADM-03	Maintain module credits and programme policy data used for GPA and warning interpretation.	Module and policy administration

FR-ADM-04	Import and export supported records in CSV format with validation feedback.	CSV data-operations module
FR-ADM-05	Record and display relevant administrative activity information.	Activity-log module
FR-ADM-06	Synchronize approved changes through Firestore so the student workflow uses current configured data.	Shared Firestore data layer

Table 3.8: Administration functional requirements

3.2.4 Non-Functional Requirements

ID	Quality	Non-functional requirement	Implemented design response
NFR-SEC-01	Security	Student and administrator privileges shall remain separated; credentials shall not be stored as readable plain text; network requests shall use HTTPS endpoints where provided.	Separate clients, hashed credential records, session controls and HTTPS-based cloud/API access.
NFR-PRI-01	Privacy	Only data required for academic prediction and support shall be requested, and wellbeing output shall not be presented as a clinical diagnosis.	Lifecycle-dependent input fields, advisory DASS-21 wording and no automated institutional data feed.
NFR-USA-01	Usability	Student results, warnings and errors shall be presented in language understandable to an undergraduate user.	Charts, labelled warnings, validation messages, guided questionnaire and conversational explanations.
NFR-REL-01	Reliability	Invalid or missing identifiers, grades and prediction values shall be rejected or reported instead of silently producing misleading output.	Validation in manual grade entry processing, GPA calculation and Flask request handling.

NFR-AVL-01	Availability	The system shall identify failures of required network or external services, although offline prediction is not required in the prototype.	Error handling and Gemini fallback; Firestore, Render and AI services remain external dependencies.
NFR-MNT-01	Maintainability	Student, administration, prediction and data concerns shall be separated so that one component can be revised without redesigning the complete system.	Android, React, Flask and Firestore responsibilities are modularized.
NFR-CFG-01	Configurability	Programme structures, batches, semesters, modules, credits and relevant policies shall be administrator-maintained.	Firestore-backed configuration controlled through the web panel.
NFR-CMP-01	Compatibility	The student interface shall operate on supported Android devices, while the administration interface shall operate through a modern web browser.	Native Android student application and browser-based React panel; no iOS client.
NFR-INT-01	Integrity	Duplicate identifiers and inconsistent academic relationships shall be validated before records are accepted.	Client-side and data-operation checks in the administration workflow.
NFR-ETH-01	Responsible use	Predictions, screening categories and chatbot responses shall be treated as decision support rather than official academic, medical or counselling decisions.	Advisory labels and separation of regression, academic rules and wellbeing guidance.

Table 3.9: Non-functional requirements

3.3 System Design

3.3.1 Design Philosophy and Constraints

- Student-centered visibility. The student receives direct access to personal results, analytics, predicted GPA, warnings, calendar support and explanations rather than depending on an administrator to interpret every record.

- Lifecycle sensitivity. A model and questionnaire are selected according to the evidence that exists at the student's current semester stage; unavailable values are not replaced by invented defaults.
- Interpretable prediction. Multiple Linear Regression was retained because a continuous GPA outcome and explainable coefficients were more appropriate to the prototype than a less transparent complex model.
- Statistical and deterministic separation. MLR produces an estimate, while attendance, continuous assessment, workload, and wellbeing conditions are evaluated through explicit rules.
- Role separation. The Android student application and web administration panel have separate interfaces and responsibilities, reducing accidental exposure of administrative controls.
- Configurable academic structure. Degree, batch, semester, module, credit and policy data are maintained as data, so the architecture is not permanently tied to one fixed programme.
- Graceful external-service handling. The chatbot has a Gemini fallback, while errors are reported when a required cloud service remains unavailable.
- Responsible support boundaries. DASS-21 and chatbot functions are limited to awareness and advisory guidance and do not replace official or professional decisions.

The following constraints directly shaped the implemented design:

- Platform boundary. The student application is Android-only; an iOS version was not implemented.
- Network dependency. Firestore, the Render-hosted Flask service, and external chatbot services require network access; offline prediction was not implemented.
- Data boundary. The ML models were developed with a 1,000-record synthetic dataset rather than a large official longitudinal Saegis Campus dataset.
- Input quality. Attendance, study hours, sleep, current marks, and wellbeing responses depend on accurate student self-reporting.
- Absence of PDF extraction. Automated PDF parsing and optical character recognition (OCR) were excluded due to the high variability and unstructured nature of institutional result PDFs. Consequently, the system relies exclusively on the manual entry of academic grades.
- Integration boundary. No official student-management system, LMS, biometric attendance, or wearable-device API was integrated.
- Security boundary. The prototype uses custom Firestore-backed account verification and session logic rather than Firebase Authentication; production deployment would require additional security hardening.

3.4 Static Architecture (High-Level System Architecture)

Figure 3.1 presents the major components and their static relationships. Undergraduate and administrator access is separated at the presentation layer. Both clients use Firestore data relevant to their role, while the Android application additionally communicates with the Flask prediction service and the external conversational-AI route. Model artifacts are maintained on the server rather than distributed in the Android package, allowing prediction logic to be revised without requiring the model itself to be embedded on every device.

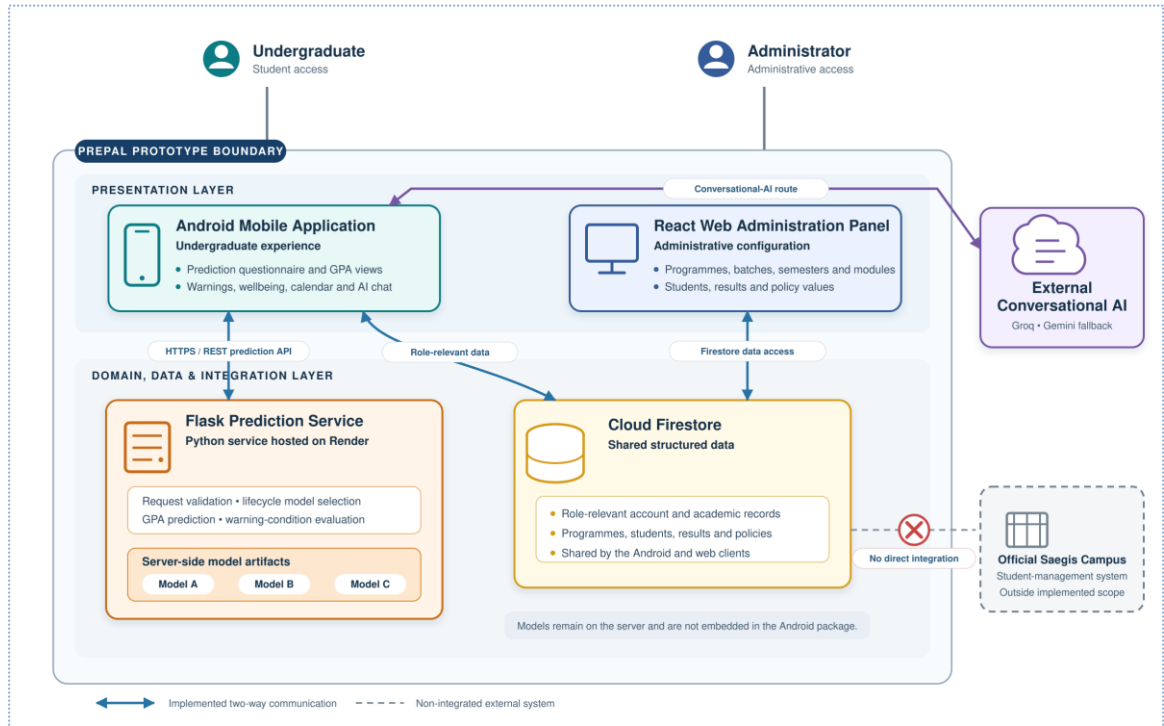


Figure 3.1: High-level static architecture of the implemented PrePal system

The architecture is intentionally not represented as a direct integration with the official Saegis Campus student-management system. Academic structures, account records, and policy values are maintained through the PrePal administration panel. This distinction prevents the prototype from claiming institutional data connectivity that was not implemented.

PrePal uses a distributed, service-oriented prototype architecture in which each technology is assigned a distinct responsibility. The Android application owns the undergraduate user experience; the React panel owns administrative configuration; Cloud Firestore provides shared structured data; the Flask service validates prediction requests, selects the applicable model, and evaluates warning conditions; and external conversational-AI services provide general academic explanations. This arrangement avoids placing administrative controls or model artifacts inside the student interface.

The design follows a layered flow even though the components are deployed separately. Presentation logic captures input and displays results; application logic validates the current user and academic context; domain logic performs GPA calculation, model selection, prediction, and rule evaluation; and data/integration logic communicates with Firestore, the Render-hosted service, and the AI providers. The layers are logical rather than a claim that every component uses the same internal package structure.

3.5 Module Design

Module	Responsibility	Inputs	Outputs	Technology
Account and Session	Verifies student or administrator access, preserves role separation, and maintains session state.	Identifier/password; account status	Authenticated role or validation error	Android, React, Firestore
Academic Administration	Maintains programmes, batches, semesters, modules, students and policies.	Administrator forms and CSV data	Validated Firestore documents and activity records	React/Vite/TypeScript
Result Entry Processing	Validates a student's manually entered grades for a given semester and maps them to modules.	Manually entered grades and student identifier	Normalized grade set or validation error	Android Java logic

GPA and CGPA Analytics	Maps grades to points, applies credit weighting and prepares performance summaries.	Grades, credits and semester records	GPA, CGPA, weak-module and trend data	Java/Python logic and charts
Lifecycle Context	Determines the active semester, available academic evidence and applicable questionnaire/model pathway.	Batch, semester status, previous results and CA availability	Model A/B/C pathway or unavailability reason	Android, Firestore, Flask
Prediction Service	Validates numerical inputs and runs the selected Multiple Linear Regression model.	Lifecycle-specific feature vector	Predicted semester GPA and technical response	Flask, scikit-learn, Render
Warning Engine	Applies attendance, CA, workload, and wellbeing checks independently of the regression estimate.	Questionnaire, policies, credits and DASS-21 categories	Structured academic and awareness warnings	Python rule logic

Calendar and Reminder	Supports academic event organization within the student application.	Event title, date/time and reminder details	Saved event and displayed reminder	Android and Firestore
AI Academic Chat	Returns general academic explanations and guidance using a primary and fallback service.	Student prompt and available academic context	Advisory chatbot response or service error	Groq with Gemini fallback
CSV and Activity Records	Supports bulk administrative data operations and accountability information.	CSV file or administrator action	Imported/exported records and log entry	React and Firestore

Table 3.10: High-level module responsibilities

3.6 Dynamic Behavior (System Workflows)

Dynamic behavior describes how the static components collaborate during an actual user task. The primary undergraduate workflow begins with authorized access, retrieves the student's configured academic context, processes manually entered results, displays retrospective analytics, collects lifecycle-appropriate prediction evidence, invokes the server, interprets the returned warnings, and then provides follow-up support functions. Figure 3.2 shows this end-to-end behavior.

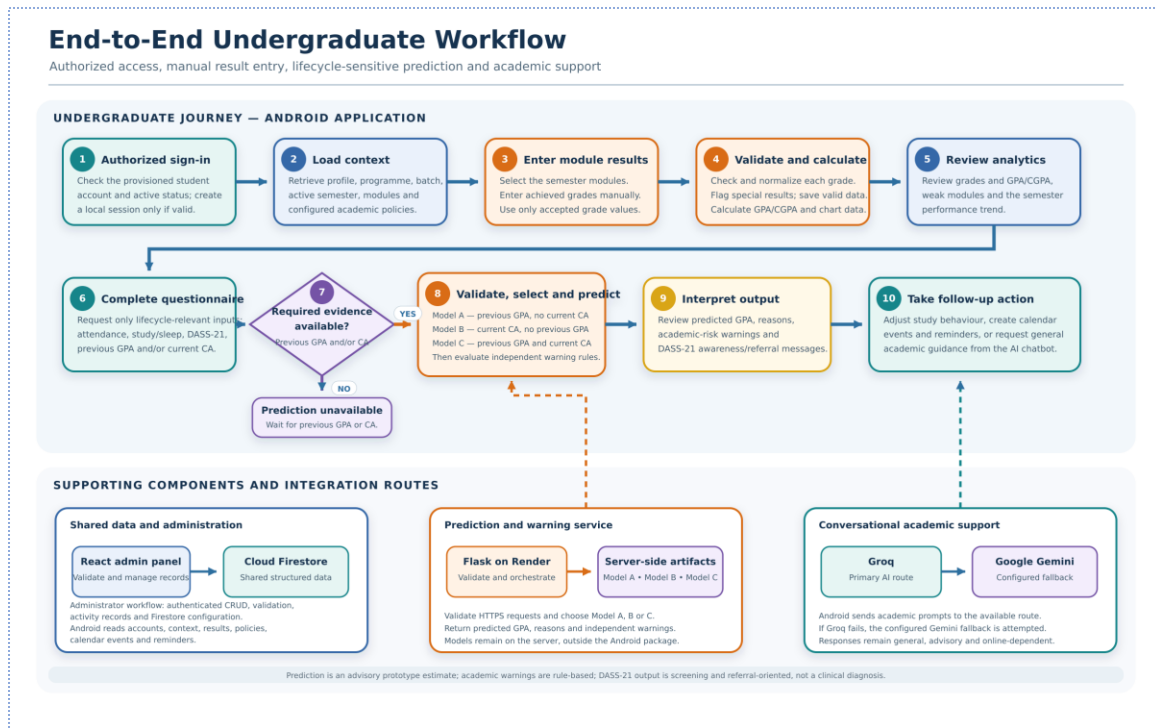


Figure 3.2: End-to-end undergraduate workflow

- **Student access workflow.** The Android application checks the provisioned account and active state, creates the local session, and retrieves the student's profile, batch, and semester context. A failed check returns an error rather than opening the dashboard.
- **Result-entry workflow.** The student manually inputs grades for their modules. Entered grades are normalized, special values are flagged, and valid results are saved to Firestore and used for GPA/CGPA and chart generation.
- **Prediction workflow.** The questionnaire requests only fields applicable to the student's stage. The Flask service validates the request, selects Model A, B or C, computes the estimate, runs independent warning rules and returns a structured response.
- **Administration workflow.** An authorized administrator selects a management area, creates or edits a record, receives validation feedback, and commits valid data to Firestore. Relevant activity information is recorded for later review.
- **Chatbot workflow.** The Android chat interface sends an academic prompt to Groq. If the primary service route fails and the configured fallback is available, Gemini is attempted. The final response remains advisory.
- **Calendar workflow.** The student creates an academic event with a date and reminder context; the application validates and displays the saved event in the calendar interface.

3.7 Data Flow Design

The data-flow design separates external actors, transformation processes, persistent data and returned outputs. Student credentials, manual grade inputs, prediction responses, calendar entries and chatbot prompts enter through the Android application. Administrators provide the academic structures and policies needed to interpret those inputs. Firestore stores shared structured records, while the Flask and conversational-AI services perform specialized processing. Figure 3.3 summarizes these flows without implying an unimplemented official-system integration.

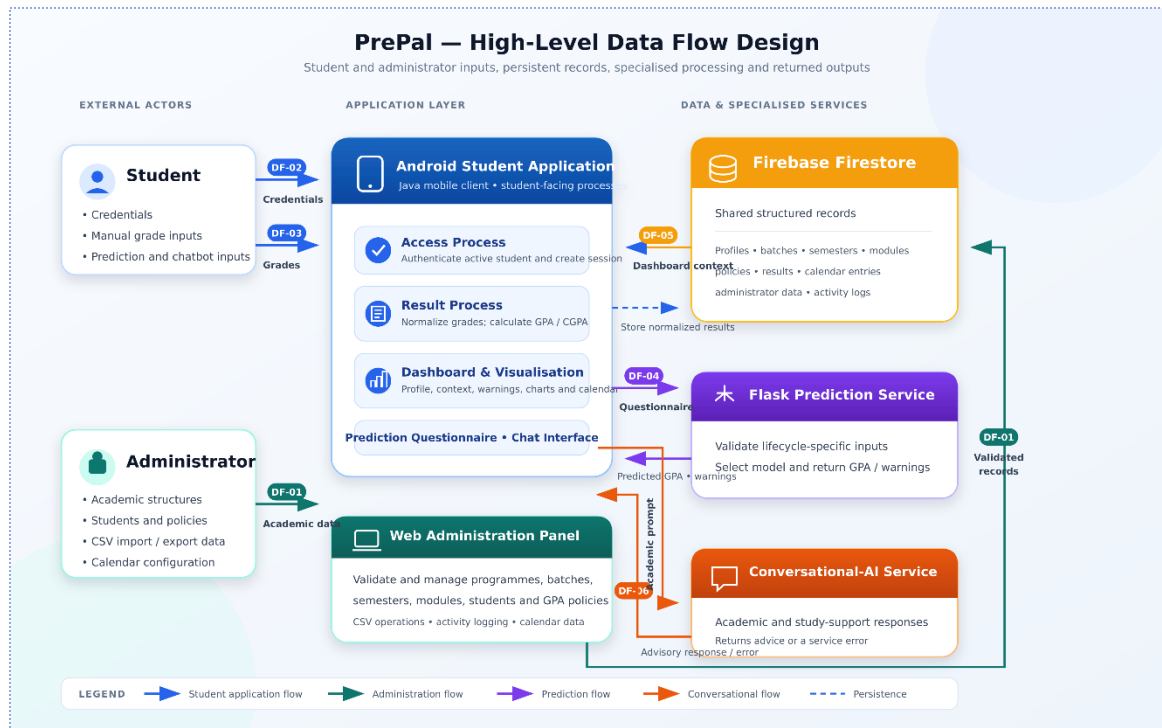


Figure 3.3: PrePal Data Flow Design

Flow	Source and destination	Data transferred	Result
DF-01	Administrator -> Administration process	Programme, batch, semester, module, student, policy and CSV data	Validated Firestore records
DF-02	Student -> Access process	Student identifier and password	Session or access error
DF-03	Student -> Result process	Manually entered grades and student identity	Normalized grades, GPA/CGPA and charts

DF-04	Student -> Prediction service	Lifecycle-specific questionnaire and semester context	Predicted GPA, warnings or missing-data message
DF-05	Firestore -> Student application	Profile, batch, semester, module, policy, result and calendar data	Student dashboard context
DF-06	Student -> Conversational service	Academic or study-support prompt	Advisory chatbot response or service error

Table 3.11: Data Flow Descriptions

3.8 Data Model (High-Level)

Cloud Firestore is a document-oriented NoSQL database. The high-level model therefore expresses logical ownership and reference relationships rather than relational tables with enforced foreign keys. Identifiers connect students to batches, batches to programmes and semesters, modules to semester structures, and academic results to both the student and semester. Administrative actions and student-owned support records are maintained separately.

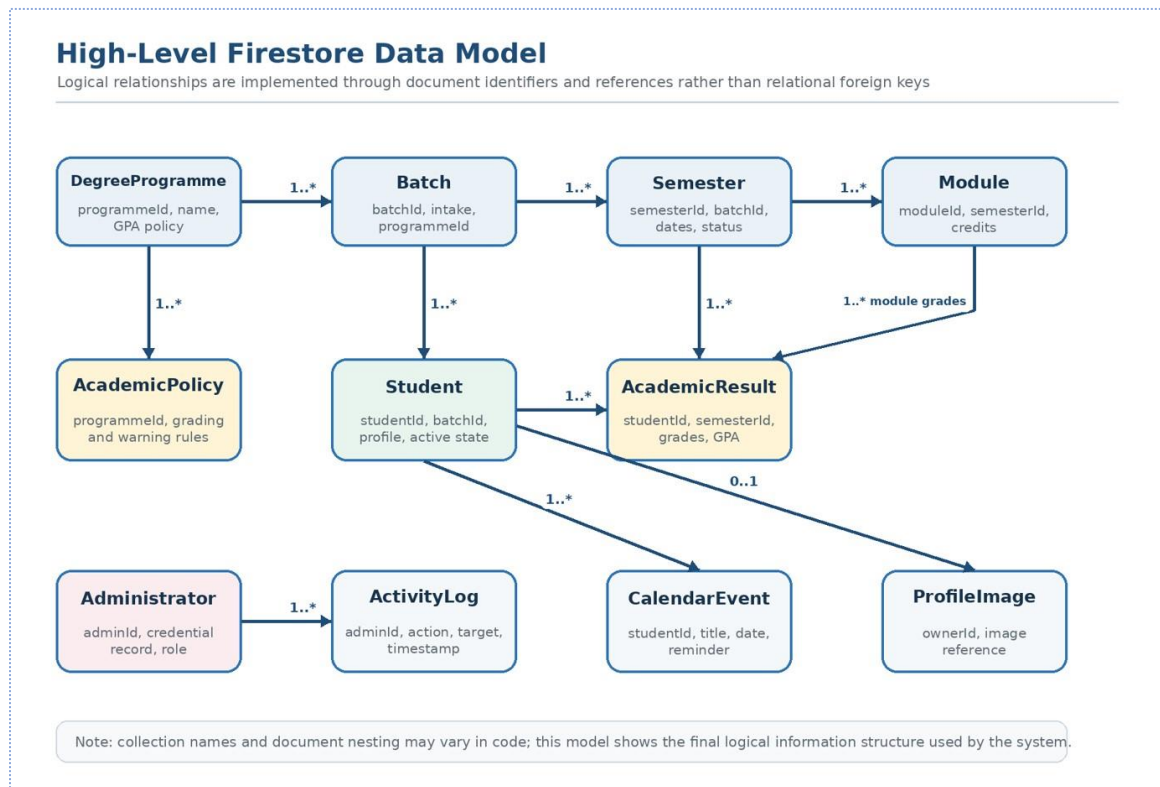


Figure 3.4: High-Level Firestore Data Model

Logical collection	Principal data	Relationship/purpose
Admins	Authorized administrator identity, credential representation and role data	Referenced by activity records
AllStudents	Student identity, name, campus details, batch reference, account state and profile data	Belongs logically to one configured batch
DegreePrograms	Programme identifier, name and programme-level academic configuration	Parent of batches and relevant policies
Batches	Batch identifier, intake information and programme reference	Parent context for students and semesters
Semesters	Academic year, semester number, dates, status and batch reference	Determines lifecycle state and module mapping
Modules	Module identifier, name, credit value and semester mapping	Used in result interpretation and GPA calculation
AcademicResults	Student/semester references, normalized grades, GPA values and result metadata	Supports CGPA and historical analytics
AcademicPolicies	Grading, attendance, CA and workload-related configuration	Used by analytics and warning interpretation
ActivityLogs	Administrator action, affected area, and time information	Supports accountability and review
CalendarEvents	Student-owned academic event and reminder information	Displayed in the Android calendar
ProfileImage	Reference to a user-associated profile image	Linked to the owning profile

Table 3.12: High-level logical data entities

3.9 Algorithms Implemented

The completed system combines deterministic academic calculations, manual entry validation logic, lifecycle routing, statistical inference and rule-based interpretation. These algorithms are kept separate because they solve different problems: GPA is an exact credit-weighted calculation, MLR is an estimate, and institutional warnings are explicit conditions.

- Algorithm 1 - Result entry and normalization. The student manually inputs grades for their modules. The system identifies the expected module-grade positions, grade

strings are normalized, and unsupported or ambiguous content produces a validation response rather than an assumed grade.

- Algorithm 2 - Grade-point mapping and special-grade handling. Recognized letter grades are mapped to the configured grade-point scale. AB, MC and NE are flagged for display and policy handling rather than incorrectly converted into ordinary numeric achievement values.
- Algorithm 3 - Semester GPA. For valid modules, semester GPA is calculated as: $\text{sum}(\text{grade point} \times \text{module credits}) / \text{sum}(\text{module credits})$. Credit weighting ensures that a module contributes in proportion to its configured academic credit value.
- Algorithm 4 - Cumulative GPA. CGPA is calculated across the student's available completed result records using the total weighted grade points divided by the total applicable credits. This avoids averaging semester GPA values without considering different credit totals.
- Algorithm 5 - Weak-module and trend preparation. Module grade points are compared and prepared for a bar-chart representation in which weak modules receive a distinct visual state. Semester GPA values are ordered chronologically to create the trend view.
- Algorithm 6 - Lifecycle and model selection. The service evaluates whether previous GPA and current mid-semester/assignment marks exist. A continuing student before CA uses Model A; a new student with CA but no previous GPA uses Model B; a continuing student with both types of evidence uses Model C; and a brand-new student with neither receives an unavailability message.
- Algorithm 7 - Multiple Linear Regression inference. The selected model estimates a continuous GPA using $y = \text{beta}_0 + \text{beta}_1x_1 + \dots + \text{beta}_nx_n$. The feature vector contains only variables defined for that model. Model development and technical evaluation use scikit-learn and the MAE, MSE and R-squared metrics.
- Algorithm 8 - Academic warning engine. After prediction, explicit conditions are checked independently. Attendance below the configured 80% threshold triggers an attendance warning. Continuous-assessment warnings use available marks and configured conditions. The workload reference is derived from programme credits and semester duration using the SLQF definition of 50 notional hours per credit.
- Algorithm 9 - DASS-21 awareness processing. The 21 responses are grouped into the depression, anxiety and stress subscales and interpreted using the implemented category rules. The result selects awareness- or referral-oriented wording and is never presented as a clinical diagnosis.
- Algorithm 10 - Chatbot primary/fallback routing. The academic prompt is first submitted through the Groq route. If that route fails and the fallback is configured, Gemini is attempted. A response is displayed only as advisory guidance; otherwise, the application reports service unavailability.

- Algorithm 11 - Custom account/session processing. The entered identifier is matched to a provisioned Firestore record, the password representation is compared with the stored hash, account state and role are checked, and an Android session is stored locally after successful verification.

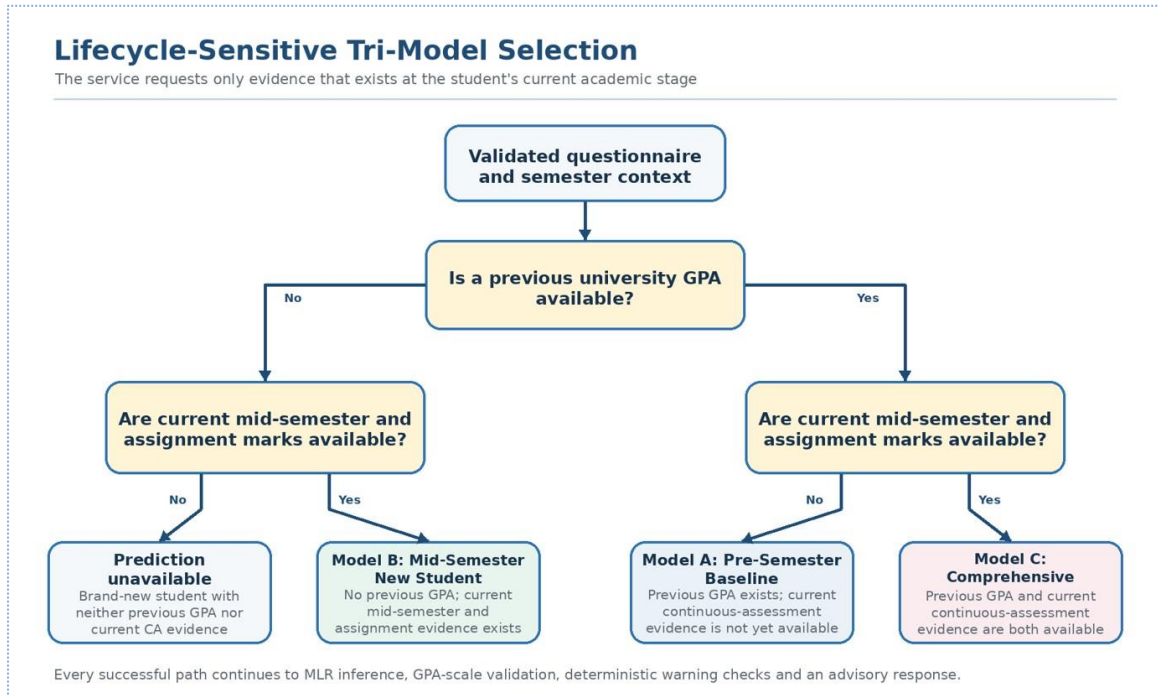


Figure 3.5: Lifecycle-Sensitive Tri-Model Selection

3.10 Design Justification

Design decision	Justification	Accepted trade-off
Native Android student application	Provides direct mobile access to results, questionnaires, charts, reminders and support using the Java/XML skills and codebase developed for the project.	No iOS client; device compatibility remains Android-specific.
Separate React web administration panel	Administrative data maintenance is more suitable to a browser-based interface with wider forms, tables and CSV operations, and keeps privileged controls away from the student app.	Requires a supported browser and network access.
Cloud Firestore	Provides a shared NoSQL data layer for the Android, web and server workflows without requiring a	Relationships and consistency checks must be enforced by application

	separately administered relational server.	logic; external availability is required.
Python Flask service on Render	Separates model logic from the mobile UI and provides a deployable HTTPS service for Android requests.	Cold starts, service availability and network latency depend on the hosting environment.
Multiple Linear Regression	Matches a continuous GPA target and remains more interpretable than a complex black-box model for an academic prototype.	Assumes linear effects and is evaluated on synthetic rather than institutional longitudinal data.
Tri-Model lifecycle architecture	Prevents one fixed form from asking for evidence that does not exist and supports different stages of the undergraduate lifecycle.	More models and routing rules must be maintained than in a single-model design.
Manual grade entry	Ensures 100% data accuracy by avoiding the extreme variability and lack of standardization in institutional result PDFs.	Requires students to manually type their grades rather than relying on automated extraction.
Hybrid ML and rule engine	Keeps statistical estimation separate from mandatory institutional thresholds and explanatory warnings.	Rules must be updated when policies change.
DASS-21 awareness integration	Connects wellbeing awareness to academic support while using a recognized structured instrument.	Screening is self-reported and non-diagnostic.
Groq with Gemini fallback	Provides immediate conversational support and an alternative route if the primary service is unavailable.	Responses can still be inaccurate and require clear advisory boundaries.
Agile iterative methodology	Allowed the design to respond to supervisor feedback, code findings and changing requirements during development.	As a single-student project, the process did not implement every Scrum role or ceremony.

Table 3.13: Design decisions, rationale and accepted trade-offs

3.11 Evolution of the Design

The final architecture is not identical to the initial proposal. Agile iteration allowed the project to preserve its central goal while changing technologies, inputs and interfaces when supervisor feedback or implementation evidence showed that the earlier design was unsuitable. The evolution is documented so that incomplete proposal items are not confused with completed functions.

Design area	Earlier concept	Final implemented design	Reason for change
Initial academic prediction concept	A simpler regression concept based mainly on historical and lifestyle information	Multiple Linear Regression with separate lifecycle-sensitive Models A, B and C	Supervisor review and the need to include several academic and behavioral predictors
Administration interface	Administration functions considered within the mobile solution	Separate React/Vite/TypeScript web administration panel	Supervisor requirement and better suitability for structured data and CSV operations
Prediction input design	One general questionnaire	Previous GPA and current CA fields are requested only when available	Avoid invalid inputs for new students and reflect semester stage
Result entry	Proposal allowed PDF upload and OCR extraction	Manual grade entry with validation rules; automated PDF extraction removed	Reduce OCR inaccuracies, address inconsistent PDF layouts, and ensure 100% data integrity
Wellbeing measure	Stress was initially represented through PSS-10/general stress values	DASS-21 depression, anxiety and stress screening with non-diagnostic messages	Late-stage evidence reviews and supervisor discussion on stress, sleep and academic support

Study-plan support	Standalone personalized study-plan output	Personalized planning and study guidance are delivered conversationally through the AI chatbot	Reuse the implemented conversational support interface instead of a separate dashboard generator
AI provider design	General chatbot concept	Groq primary service with Gemini fallback	Improve service continuity while keeping responses advisory
Academic context	Fixed or limited programme examples during development	Campus-wide undergraduate architecture with programme-specific administrator configuration	User requirement that PrePal be designed for undergraduates across Saegis Campus programmes
Proposal functions not completed	Prediction confidence indicator and past-paper access were proposed	Excluded from the final system and recorded as limitations/future work	Fixed final-year project time and priority given to the integrated prediction, warning and administration workflow

Table 3.14: Evolution from proposal concepts to the final implemented design

The practical Agile cycle used across these changes was: identify or revise a requirement; design a small functional increment; implement it in the relevant Android, web, server or data component; review the increment using supervisor feedback and technical observations; correct defects or adjust scope; and integrate the accepted increment into the working system. This cycle was repeated for academic structures, GPA calculations,

prediction variables, model routing, result processing, wellbeing support, administration and deployment.

Agile increment	Focus	Resulting project output
Increment 1	Problem, proposal and evidence base	Problem definition, literature review, survey and initial functional scope
Increment 2	Core student and data foundation	Android account, profile, Firestore structures and academic-record foundation
Increment 3	Academic administration	Programme, batch, semester, module and student management; later moved to the web panel
Increment 4	Prediction model refinement	MLR dataset, feature review, evaluation metrics and Tri-Model lifecycle logic
Increment 5	Result and analytics workflow	Manual grade entry processing, GPA/CGPA correction, weak-module chart and trend view
Increment 6	Rules and wellbeing	Attendance, CA and SLQF workload warnings; DASS-21 awareness integration
Increment 7	Support and integration	Calendar/reminders, Groq chatbot with Gemini fallback, cloud integration and error handling
Increment 8	Final refinement	Technical testing, corrected scope, deployment, limitations and final documentation

Table 3.15: Agile increments reflected in the completed PrePal project

This evolution demonstrates controlled scope adaptation rather than uncontrolled feature addition. The final system concentrates on the functions that were actually completed and technically integrated. Past-paper access, prediction-confidence indicators, iOS support, offline prediction, and direct official-system integration remain outside the implemented design and are therefore treated as limitations or future work rather than requirements satisfied by the prototype.

4. Implementation

4.1 Introduction

This chapter explains how the approved PrePal design was converted into a working integrated prototype. It describes the implementation of the Android student application, the React web administration panel, the Python Flask processing and prediction service, the Cloud Firestore data layer and the connections between them. The emphasis is on completed implementation decisions, validation behavior and deployment boundaries rather than repeating the requirements and architecture already specified in Chapter 3.

Implementation followed the Agile, feedback-driven approach described earlier. Small functional increments were built, demonstrated and corrected across the mobile, web, server and cloud-data components. Supervisor observations and technical findings led to several material refinements, including the move to a browser-based administration panel, correction of the CGPA calculation, lifecycle-dependent Model A/B/C routing, replacement of the earlier stress method with DASS-21 awareness processing, and the final manual grade entry workflow without automated PDF/OCR extraction.

The completed prototype is distributed across independently deployable technologies. Java and XML implement the undergraduate interface on Android; React, Vite and TypeScript implement administrative screens in a web browser; Firestore stores the shared academic and account records; and the Render-hosted Flask service executes model selection, Multiple Linear Regression inference and warning rules. Groq is used for primary conversational support with a Gemini fallback. No direct connection to the official Saegis Campus student-management system or LMS was implemented.

Component	Technology	Implemented responsibility	Execution location
Student frontend	Java and XML for Android	Student access, profile, manual result entry, GPA/CGPA views, charts, questionnaire, warnings, DASS-21, calendar and chatbot	Android device
Administration frontend	React, Vite and TypeScript	Programme, batch, semester, module, student and policy management; CSV operations; activity views	Web browser / hosted web panel
Shared data layer	Cloud Firestore	Structured account, academic, result, policy, calendar and activity records	Firebase managed cloud
Processing and prediction	Python, Flask, and scikit-learn	Input validation, lifecycle routing, MLR inference and rule evaluation	Render-hosted service

Conversational support	Groq with Gemini fallback	General academic explanations, study guidance and non-clinical wellbeing support	External AI services
------------------------	---------------------------	--	----------------------

Table 4.1: Technologies and their implementation responsibilities

Implementation boundary: The chapter describes the working prototype only. It does not claim iOS support, offline prediction, automated PDF/OCR extraction, past-paper access, a prediction-confidence indicator, a standalone automated study-plan dashboard or direct institutional-system integration.

4.2 Frontend Implementation

PrePal has two separate frontends because the student and administrator tasks differ substantially. The Android application presents a guided, personal academic workflow suitable for a mobile device. The web administration panel presents wider forms, tables and bulk-data operations suitable for browser use. Keeping these interfaces separate also prevents administrative controls from being distributed inside the student application.

4.2.1 Android Student Application

The student application was developed natively in Android Studio using Java for application logic and XML for layouts. Activities and related interface elements were organized around the principal student tasks: authentication, profile and context loading, manual result entry, academic analytics, GPA prediction, wellbeing awareness, calendar support and conversational assistance. Navigation returns the student to the appropriate dashboard or previous screen rather than exposing administration functions.

Interface validation is applied close to the point of data entry. Required fields are checked before a request is submitted, numeric inputs are restricted to the expected ranges, unavailable academic fields are not requested, and network or validation errors are returned as user-readable messages. The Android interface therefore acts as the first validation layer, while the server repeats authoritative validation before prediction or result submission is executed.

4.2.2 Student Access, Profile and Session Handling

Student accounts are provisioned through the administrative workflow. During sign-in, the Android application retrieves the matching Firestore student record, compares the entered password representation with the stored SHA-256 hash, verifies that the account is active and confirms that the record belongs to the student access path. A failed identifier, password or account-state check produces an error and does not open the student dashboard.

After successful verification, the application stores the required local session information through SharedPreferences. The stored session is used to preserve the signed-in state and the student identifier needed for later Firestore reads. Signing out clears this local state. The profile workflow displays the student's configured identity and academic context and

supports the implemented profile-image reference without exposing another student's records.

4.2.3 Manual Result Entry and Academic Analytics

The result workflow begins with the manual grade entry interface. The student manually inputs their module grades, which are then passed to the configured validation route. The interface displays real-time feedback and distinguishes successful grade inputs from unsupported formats or ambiguous grade strings. Because automated PDF extraction was removed from the final implementation to ensure 100% data integrity, any incorrect entries must be manually corrected by the student before submission.

When normalized grades are returned, the application displays the module results and the calculated semester GPA. Available result records are combined using credit weighting to produce CGPA. Weak-module information is prepared for a bar-chart view, while semester GPA values are ordered to create the performance-trend view. Special values such as AB, MC and NE are displayed as exceptional academic states rather than incorrectly converted into ordinary grade points.

4.2.4 Lifecycle Questionnaire, Prediction and Wellbeing Interface

The questionnaire is generated according to the student's academic stage instead of presenting one fixed form. Firestore semester status, previous-result availability and the presence of current mid-semester and assignment marks determine which academic evidence can be requested. Common behavioural fields include attendance, weekly study hours and daily sleep duration. Previous GPA and continuous-assessment fields appear only when those values are available for the applicable lifecycle pathway.

DASS-21 responses are collected through the implemented questionnaire and are processed separately from the regression estimate. The interface presents depression, anxiety and stress awareness or referral-oriented messages using non-diagnostic wording. Prediction output is similarly labelled as an advisory semester-GPA estimate. The result view combines the prediction with contributing-factor explanations and separately triggered academic warnings so that a statistical estimate is not confused with an institutional rule.

4.2.5 Calendar, Reminders and AI Chat

The calendar module allows a student to create, view and manage personal academic events and reminder information. Event details are associated with the signed-in student and stored through the shared data layer. These functions support organization but are not automatically synchronized with the official Saegis timetable or LMS.

The AI chat interface implemented in the Android application accepts a student's academic or study-support prompt and displays the returned response in a conversational view. The primary route uses Groq; Gemini is attempted when the primary service fails and the fallback is available. Prompts and interface wording restrict the feature to general academic

explanations, study guidance and non-clinical support. The interface reports service unavailability instead of presenting a fabricated response when neither route succeeds.

4.2.6 React Web Administration Panel

The administration frontend was implemented as a React application created with Vite and TypeScript. It uses a separate administrator access path and browser-oriented page structure. Reusable forms, validation messages, lists and management views support degree programmes, batches, semesters, modules, students, academic policies and result-related administration. The semester interface includes start and end dates together with active, completed and inactive status values needed by the student lifecycle logic.

Administrative forms validate required identifiers and values before committing changes. Duplicate identifiers, especially StudentID values, are rejected, and related records are checked before they are accepted. The web panel also provides the implemented CSV import/export functions, dashboard information, administrator profile functions and activity-record views. Successful changes are written to Firestore so the student application and server use the current administrator-maintained configuration.

Frontend area	Implementation behavior	Visible result
Student sign-in	Verify provisioned identifier, password, role and active state	Session or readable access error
Student dashboard/profile	Load identity, programme/batch and current semester context	Personal academic context and navigation
Manual result entry	Manually input module grades and submit them for validation	Normalized grades or validation message
Academic analytics	Display GPA, CGPA, weak modules and semester trend	Retrospective performance information
Prediction questionnaire	Show only lifecycle-relevant academic and behavioral inputs	Validated request for Model A, B or C
Prediction result	Present estimated GPA, reasons and independent warnings	Advisory decision-support output
DASS-21	Collect 21 responses and present non-diagnostic messages	Awareness or referral-oriented guidance
Calendar and reminders	Maintain student-owned academic events	Saved personal organization data
AI chat	Submit a prompt and display primary or fallback response	General advisory academic guidance
Web administration	Maintain academic structures, users, policies, CSV data and logs	Validated shared configuration

Table 4.2: Implemented frontend areas and visible behavior

4.3 Backend Implementation

The backend separates processing that would be difficult to maintain safely inside the Android package. The Flask service receives structured prediction requests and manually entered grade data, performs validation, calls the applicable domain logic and returns structured success or error information. The service is deployed through Render, allowing the model artifacts and processing rules to be updated on the server without embedding a new model file in every Android installation.

4.3.1 Flask Service and Request Validation

The Flask application exposes the service routes required by the Android client. Prediction requests contain the student's current academic context and the feature values available at that lifecycle stage. Result-processing requests contain the manually entered grade strings and the student identifier needed to map the results. The service checks that required values exist, converts accepted numerical fields to the expected types and rejects out-of-range, incomplete or inconsistent data before invoking domain logic.

Errors are returned as explicit validation or service messages rather than hidden exceptions. This distinction allows the Android application to explain whether a prediction is unavailable because academic evidence is missing, the manual entry format is unsupported, an input is invalid or the hosted service cannot currently be reached.

4.3.2 Grade Validation and Processing

Because automated PDF extraction was removed from the final implementation, the processing logic relies entirely on the grades manually inputted by the student. The processor accepts the submitted grades, maps them to the configured module positions, and checks them against the supported result layout for the semester.

Manually entered grade strings are normalized before academic calculations. Valid grades are mapped to their configured grade points; AB, MC and NE remain flagged special values. The processor does not assume a grade when the input format is invalid or ambiguous. Instead, it returns a validation response so the student can manually correct the erroneous entry.

4.3.3 GPA and CGPA Processing

The semester GPA implementation multiplies each valid module grade point by its configured credit value, totals the weighted points and divides by the applicable credits. CGPA uses the same principle across all available completed result records: total weighted grade points are divided by total applicable credits. This corrected implementation avoids the earlier error of treating semester GPA values as if every semester necessarily had the same credit total.

The analytics processor prepares a normalized module result set for the weak-module chart and an ordered list of semester GPA values for the trend view. Calculated values and

supporting result metadata are stored in the academic-result structure so the mobile interface can reconstruct the student's historical performance view.

4.3.4 Tri-Model Multiple Linear Regression

Three Multiple Linear Regression artifacts were implemented using scikit-learn because the evidence available to an undergraduate change over time. Model A supports a continuing student for whom previous GPA exists but current continuous-assessment marks are not yet available. Model B supports a student with current mid-semester and assignment marks but no previous university GPA. Model C uses both previous GPA and current continuous-assessment evidence. Behavioural inputs required by the selected artifact are added to the lifecycle-specific academic evidence.

The models were developed using a 1,000-record synthetic dataset designed around the selected academic and behavioural variables. The implementation used a train/test split and evaluated the held-out predictions through R-squared, Mean Absolute Error and Mean Squared Error. The resulting artifacts are loaded on the server and are treated as prototype estimation models rather than institutionally validated official predictors.

Simplified lifecycle router:

```
python
if previous_gpa_available and current_ca_available:
    use Model C
elif previous_gpa_available:
    use Model A
elif current_ca_available:
    use Model B
else:
    return unavailable_message
```

After routing, the service constructs the feature vector expected by the selected artifact, performs inference and validates the returned value against the GPA scale used by the application. The response identifies the model pathway and returns the estimate with the explanatory and warning information required by the mobile interface.

4.3.5 Academic Warning and DASS-21 Logic

The warning engine is deliberately independent of the regression equation. Attendance is compared with the configured 80% requirement; available continuous-assessment evidence is checked against the applicable configured conditions; and the workload reference is derived from programme credits, the SLQF definition of 50 notional learning hours per credit and the semester duration. Separating these rules prevents an acceptable predicted GPA from suppressing a mandatory academic warning.

DASS-21 processing groups the 21 questionnaire responses into the depression, anxiety and stress subscales and applies the implemented category boundaries. The selected output message is awareness- or referral-oriented and is not incorporated as a clinical diagnosis. The service keeps the statistical estimate, academic rules and wellbeing screening logically distinct even when the mobile result screen presents them together.

4.3.6 Conversational-AI Routing

The conversational support logic submits the student's academic prompt through the configured Groq route. When the primary route fails and the Gemini fallback is configured, the request is attempted through Gemini. Responses are returned to the Android chat interface with advisory boundaries. Timeouts, provider errors and unavailable fallback service are converted into understandable failure messages rather than silently reusing an unrelated answer.

Backend responsibility	Principal input	Returned result
Account/context validation	Identifier, role, status and lifecycle values	Accepted context or validation error
Grade processor	Manually entered grades and student identifier	Normalized grades or validation error
Academic analytics	Grades, credits and historical results	GPA, CGPA and chart-ready data
Lifecycle router	Previous-GPA and current-CA availability	Model A, B, C or unavailable pathway
MLR inference	Model-specific feature vector	Estimated semester GPA
Academic warning engine	Attendance, CA, workload and configured policies	Structured academic warnings
DASS-21 processor	Twenty-one self-reported responses	Non-diagnostic awareness/referral messages
AI router	Academic support prompt	Groq response, Gemini fallback or error

Table 4.3: Backend processing responsibilities

4.4 Database Implementation

Cloud Firestore was implemented as the shared document-oriented database for the Android application, React administration panel and supporting service logic. Its NoSQL structure allows each academic area to be stored as a collection of documents, while identifiers provide the logical relationships that would otherwise be implemented through relational foreign keys. Because Firestore does not enforce those relationships automatically, the Android, web and server components validate references before using them.

4.4.1 Firestore Collections

Logical collection	Implemented data	Principal use
Admins	Administrator identity, credential hash and role information	Administrator access and activity ownership
AllStudents	Student identity, campus details, batch reference, account state and profile data	Student access and academic context
DegreePrograms	Programme identifier, name and programme-level configuration	Parent configuration for batches and policies
Batches	Batch identifier, intake data and programme reference	Connects students to programme and semester context
Semesters	Academic year, semester number, dates, status and batch reference	Determines lifecycle state and active context
Modules	Module identifier, name, credits and semester mapping	Grade interpretation and GPA weighting
AcademicResults	Student/semester references, grades, GPA and result metadata	Historical analytics and CGPA
AcademicPolicies	Grading, attendance, CA and workload configuration	Academic calculations and warning interpretation
ActivityLogs	Administrator action, affected area and time information	Administrative accountability and review
CalendarEvents	Student-owned event and reminder information	Android calendar and reminder views

ProfileImage	Reference to a user-associated profile image	Student or administrator profile presentation
--------------	--	---

Table 4.4: Implemented Firestore collections

4.4.2 Logical References and Lifecycle Data

Student documents contain the batch identifier needed to locate the correct programme and semester context. Batch documents reference their degree programme, semester documents reference the applicable batch, and module documents contain the mappings and credit values required for result interpretation. Academic-result documents reference both the student and semester, so completed records can be ordered and combined for CGPA and trend calculations.

Lifecycle routing depends on more than a static student record. Semester start/end dates and active, completed or inactive status values determine which context is current; previous academic results show whether a previous GPA exists; and the questionnaire records whether current mid-semester and assignment evidence is available. These values allow the mobile and Flask logic to avoid requesting inputs that do not yet exist.

4.4.3 Data Validation, Activity Records and Integrity

The administration panel validates identifiers before a new record is committed. Duplicate StudentID values and duplicate academic identifiers are rejected, required relationships are checked, and numeric or date values are validated. Where removal would damage related records, the implemented workflow can use inactive states rather than treating every administrative action as an unrestricted physical deletion.

Relevant administrative changes create activity information containing the responsible administrator, affected area, and time context. CSV imports pass through validation before accepted rows are written. Invalid rows are reported so one malformed entry does not silently create inconsistent academic structures.

4.4.4 Credential and Access Data

The prototype uses custom Firestore-backed account verification rather than Firebase Authentication. Passwords are represented by SHA-256 hashes rather than stored as readable text, application logic checks role and account state, and the Android client stores only the local session data needed for continued use. Firestore access rules and role-aware application paths provide additional separation between student and administration operations.

This implementation is suitable for the academic prototype but is not presented as production-ready institutional identity management. A full deployment would require stronger password-protection techniques, formal security testing, centrally governed authentication, credential recovery controls, and institutional privacy approval.

4.5 Integration of System Components

Component integration was performed incrementally after each client or service could complete its local responsibility. Firestore provides the shared reference point for student, programme, semester, module, policy, result and calendar data. The Android client additionally invokes the Render-hosted Flask service for manual result entry processing and GPA prediction and uses the conversational-AI routes for on-demand guidance. The React panel updates the configuration consumed by those student workflows.

Integration interface	Information exchanged	Implemented handling
Android <-> Firestore	Account, profile, programme/batch/semester context, results and calendar records	Listener/task success updates the interface; failure displays a data or network message
React panel <-> Firestore	Academic structures, students, policies, CSV records and activity data	Client validation precedes writes; duplicate or inconsistent records are rejected
Android <-> Flask	Prediction fields, lifecycle context, manually entered grades and student identifier	Structured prediction/result response or explicit validation/service error
Flask <-> model artifacts	Validated model-specific feature vector	Model A/B/C inference without distributing artifacts to the mobile package
Android/AI route <-> Groq	Academic or study-support prompt	Primary advisory response or provider failure
AI route <-> Gemini	Prompt after an eligible primary-route failure	Fallback advisory response or final unavailability message

Table 4.5: Implemented component integrations

4.5.1 End-to-End Student Integration

- I. The Android application verifies the provisioned student account and creates the local session.
- II. The application retrieves the student's profile, programme, batch, active semester, module and policy context from Firestore.
- III. The student manually inputs module grades into the interface and submits them to the validation route.
- IV. Normalized grades, calculated GPA/CGPA, and chart-ready values are stored and displayed through the student analytics workflow.

- V. The questionnaire displays only the previous-GPA and current-CA fields permitted by the loaded lifecycle context, together with the implemented behavioural and DASS-21 inputs.
- VI. The Flask service revalidates the request, selects Model A, B or C, performs inference and evaluates independent academic and wellbeing conditions.
- VII. The Android result interface presents the estimated GPA, reasons, academic warnings and non-diagnostic wellbeing messages.
- VIII. The student can create reminders or calendar events and use the chatbot for general follow-up guidance.
- IX. When required evidence or a network service is unavailable, the workflow returns a specific explanation instead of an unsupported prediction.

4.5.2 Administration-to-Student Integration

Administrator-maintained records are the source of the academic context used by the student application. A programme must contain the relevant batches; a batch must have semester records with appropriate dates and statuses; and modules and credits must be mapped before GPA and workload logic can operate correctly. Student records must reference the correct batch and remain active before access is granted.

This integration is intentionally internal to PrePal. The web panel does not synchronize with an official Saegis database, and the Android application does not assume that an external institutional record is current. The reliability of programme-wide operation therefore depends on authorized administrators maintaining accurate data in the PrePal collections.

4.5.3 Deployment Integration

The Android project is built as the student application package, while the React administration panel is deployed for browser access. The Flask application and server-side model artifacts are deployed through Render, and Firestore provides the cloud database used by both client interfaces. Endpoint and provider configuration connects the deployed components without packaging service credentials or model artifacts as ordinary visible student-interface content.

All cloud-dependent functions require network access. Render cold starts, Firestore availability, API quotas and conversational-provider outages can affect response time or temporary availability. Error handling and the Gemini fallback reduce some failures, but the implemented prototype does not provide an offline prediction or local model-execution path.

4.6 Testing and Debugging During Implementation

Testing was performed throughout the Agile implementation increments rather than postponed until all components were complete. Each function was checked first within its own component and then through the integrated workflow. The emphasis was functional correctness, validation, model evaluation and error handling. The project does not claim a complete enterprise-scale automated regression suite or a longitudinal institutional effectiveness study.

4.6.1 Frontend and Validation Testing

Android testing covered valid and invalid student sign-in, inactive accounts, session persistence and sign-out; profile and academic-context loading; supported and unsupported manual grade entries; chart and GPA/CGPA display; lifecycle-dependent questionnaire fields; input-range errors; prediction-unavailable messages; calendar operations; chatbot success/fallback behavior; and network-failure messages. Interface checks also confirmed that administration controls were absent from the student application.

Web-panel testing covered administrator access, create/update/deactivate workflows, required fields, duplicate identifiers, semester dates and statuses, module-credit values, CSV validation and activity records. Administrative changes were then checked from the student side to confirm that the updated Firestore context was being retrieved.

4.6.2 Google Colab Model Training and Testing

The Tri-Model Multiple Linear Regression logic was initially developed, trained, and tested within a Google Colab environment before integration into the Flask backend. A 1,000-record synthetic dataset designed around the selected academic and behavioral variables was imported into Colab to simulate various student lifecycles. Utilizing Python, Pandas, and Scikit-learn within the Colab notebooks, the data was pre-processed and split into training and testing subsets.

The models (Model A, B, and C) were trained independently. The interactive Colab environment allowed for rapid iteration, enabling the fine-tuning of coefficients, correlation reviews to detect implausible feature behaviour, and verification of the logic routing. The held-out test predictions were evaluated using R-squared (R^2) to measure explained variation, Mean Absolute Error (MAE) to measure the average absolute GPA error, and Mean Squared Error (MSE) to increase the effect of larger errors. Once the model performance and routing logic were verified in Colab, the final model artifacts were exported for deployment on the Render-hosted Flask service.

4.6.3 Backend and API Testing

Flask requests were checked with complete, missing, incorrectly typed and out-of-range values. All four lifecycle combinations were exercised: previous GPA without CA, CA without previous GPA, both types of evidence and neither type. The last combination was expected to return an explanation rather than invoke a model. Grade entry validation tests

included checking missing student identifiers, invalid numeric formats, special grades (AB, MC, NE), and incomplete grade submissions.

4.6.4 Database and Integration Testing

Firestore tests checked successful and failed reads/writes, required document identifiers, duplicate StudentID prevention and consistency of programme-batch-semester-module references. Academic-result records were checked across multiple semesters to confirm credit-weighted CGPA and chronological trend preparation. Calendar and activity records were checked against their owning student or administrator context.

Integration testing followed the end-to-end path from account verification to context loading, result entry, analytics, questionnaire generation, server prediction, warning display and support actions. Service errors were deliberately handled at component boundaries so a failed Render, Firestore or AI request did not produce an unhandled mobile or web state.

4.6.5 Principal Debugging Corrections

Observed issue	Debugging correction	Implemented result
Duplicate StudentID values could be accepted	Added duplicate-identifier validation before committing a student record	One provisioned identity map to one student record
CGPA logic could treat semester GPA values equally	Changed the calculation to total weighted grade points divided by total applicable credits	Correct credit-weighted cumulative GPA
One questionnaire could request unavailable evidence	Loaded lifecycle context and conditionally displayed previous-GPA and CA fields	Valid Model A/B/C inputs for different student stages
Model routing did not consistently check CA availability	Tested all previous-GPA/current-CA combinations and corrected both decision branches	Model A, B, C or unavailable response is selected consistently
Initial prediction design used a simpler regression concept	Reworked the dataset and training code in Colab for Multiple Linear Regression and three artifacts	Tri-Model lifecycle prediction service
Study-hour values did not consistently reflect the project workload interpretation	Revised the synthetic distribution and SLQF-derived checks while retaining realistic edge cases	More defensible workload feature and warning logic
Automated PDF extraction risked severe data inaccuracies due to	Removed PDF extraction and implemented validated manual grade entry	100% data integrity through a single validated entry path

unstandardized document layouts		
Invalid user inputs and special grades could break numeric processing	Added input normalization and explicit AB/MC/NE handling	Clear result calculation or validation error
Stress-only input did not provide the final wellbeing structure	Implemented DASS-21 subscale processing and non-diagnostic output wording	Depression, anxiety and stress awareness messages
Administrative mobile controls were unsuitable	Implemented the separate React web panel and retained role-separated access	Browser-based structured administration
Primary chatbot service could fail	Added Gemini fallback and final service-unavailable handling	More resilient but still advisory conversational support
Network failures could leave unclear interface states	Added progress, validation and failure messages at Firestore, Flask and AI boundaries	Understandable recovery or retry guidance

Table 4.6: Principal implementation defects and corrections

4.6.6 Testing Boundaries

The completed checks demonstrate technical operation of the principal Android, web, Firestore, manual grade entry, prediction and external-service workflows. They do not establish clinical validity for DASS-21 use, official accuracy of a predicted GPA or long-term improvement in student outcomes. The models were not trained and longitudinally validated using a large official Saegis Campus dataset, and the grade entry forms were not tested with every possible edge case across all programme types.

Before production deployment, PrePal would require a broader automated test suite, penetration and privacy testing, usability testing across all degree programmes, real institutional-data validation, controlled model monitoring and formal acceptance by academic and student-support authorities. Within the final-year project boundary, iterative testing and debugging were used to deliver a coherent prototype whose implemented features, limitations and failure conditions are explicitly documented.

Implementation outcome: The final prototype integrates the Android student workflow, React administration panel, Firestore records, Render-hosted Flask service, Tri-Model MLR artifacts, manual grade entry processing, rule-based warnings, DASS-21 awareness, calendar support and Groq/Gemini chatbot routing. Unimplemented proposal items remain recorded as limitations rather than being represented as completed functionality

5. Results and Evaluation

5.1 Introduction

This chapter evaluates the completed PrePal prototype against the project goals and the requirements defined in the preceding chapters. The evaluation considers functional correctness, interaction between the Android application, React administration panel, Cloud Firestore, Flask prediction service and external conversational-AI routes, together with the technical evaluation of the Tri-Model Multiple Linear Regression component. The purpose is to establish what the implemented system demonstrably achieves and where the available evidence remains limited.

The evaluation is intentionally bounded by the evidence produced during the final-year project. Principal workflows were tested technically and corrected during iterative development. However, the system was not evaluated through a longitudinal institutional study, production-scale load test or large usability study involving students from every Saegis Campus degree programme. Model results were produced from a 1,000-record synthetic dataset and therefore indicate prototype performance rather than validated institutional predictive accuracy.

Evaluation boundary: A successful technical test shows that an implemented function behaves as designed under the tested condition. It does not prove that a predicted GPA is an official academic outcome, that DASS-21 output is a clinical diagnosis, or that the prototype will improve long-term student performance.

5.2 Achievement of Project Goals

The project aim was to develop an integrated academic prediction and support system for Saegis Campus undergraduates. This aim was substantially achieved as a working prototype comprising a native Android student application, a React web administration panel, a shared Firestore database and a Render-hosted Flask service. The final system combines retrospective result analysis, lifecycle-sensitive GPA estimation, academic warning rules, DASS-21 awareness messages, calendar support and conversational academic guidance. The system remains a prototype because its machine-learning models use synthetic data and it is not directly connected to official institutional information systems.

No.	Project objective	Implemented evidence	Evaluation
1	Provide an Android academic-support interface	Implemented student sign-in, profile, analytics, prediction, warnings, DASS-21, calendar, reminders and AI chat.	Achieved within Android scope
2	Process manually entered results and produce academic analytics	Implemented validated manual grade entry with format normalization, GPA/CGPA, weak-module and trend views.	Achieved with 100% data integrity
3	Develop lifecycle-sensitive MLR prediction	Implemented Model A, Model B and Model C with routing based on previous-GPA and current-CA availability.	Technically achieved; institutional validity not established
4	Deploy the prediction and processing service	Implemented Flask validation, model loading, inference, rule evaluation and structured responses on Render.	Achieved subject to network/service availability
5	Provide academic-risk and wellbeing feedback	Implemented attendance, CA and SLQF-derived workload warnings plus separate DASS-21 awareness/referral messages.	Achieved as advisory support
6	Provide centralized academic administration	Implemented the React/Vite/TypeScript panel for programmes, batches, semesters, modules, students, policies, CSV operations and logs.	Achieved within PrePal-managed data
7	Integrate role-specific clients and shared data	Connected Android and web workflows to Firestore and verified the principal account, academic-data and service paths.	Achieved at prototype integration level
8	Provide conversational academic assistance	Implemented Groq-based chat with Gemini fallback for general academic and non-clinical guidance.	Achieved with external-service limitations

Table 5.1: Achievement of the principal project objectives

5.2.1 Overall Goal Assessment

The strongest achievement is the integration of several forms of student support into one workflow without representing them as the same type of evidence. GPA and CGPA are deterministic calculations from grades and credits; the future GPA is a statistical estimate; academic warnings are rule-based interpretations; and DASS-21 output is non-diagnostic screening guidance. The interface can present these results together while retaining their different meanings.

The design goal of supporting Saegis Campus undergraduates is reflected in the programme, batch, semester, module and policy configuration available through the administration panel. Nevertheless, programme-wide readiness depends on administrators configuring each degree accurately and on students accurately inputting their manual grades based on their programme's module structure. Accordingly, the architecture is institution-wide in intention, while the completed technical evaluation remains limited to the configured prototype data and the manual grade validation rules.

5.3 Testing Strategy

Testing followed the iterative Agile implementation process. A feature was checked within its own component, corrected when necessary, and then exercised through the relevant integration path. The strategy combined positive tests, negative tests, boundary-value tests and end-to-end workflow tests. Model evaluation was treated separately from interface correctness because a functioning API call does not by itself demonstrate prediction quality.

Test level	Scope	Purpose
Component testing	Individual Android activities, React views, Flask functions and Firestore operations	Confirm local behaviour before integration
Functional testing	Valid, invalid, missing and boundary inputs for each user function	Compare visible behaviour with the functional requirement
Integration testing	Android-Firestore, React-Firestore, Android-Flask, Flask-model and AI-provider paths	Confirm correct data exchange and error handling
Model evaluation	Held-out portion of the 1,000-record synthetic dataset	Calculate R-squared, MAE and MSE for each MLR model
Manual grade entry testing	Valid numeric strings, special grades (AB, MC, NE), incomplete rows, and invalid format structures	Verify format normalization, structural validation, and controlled rejection

Regression testing	Re-run affected workflows after corrections	Confirm that fixes did not break established functions
Formative usability review	Task walkthroughs and supervisor/development feedback	Identify navigation, input, wording and recovery problems

Table 5.2: Testing strategy applied to the prototype

5.3.1 Test Data and Evaluation Conditions

Functional tests used configured student, programme, batch, semester, module, policy and result records in the PrePal database. Prediction tests covered the four possible combinations of previous-GPA and current continuous-assessment availability so that Model A, Model B, Model C and the no-prediction-yet outcome could each be checked. Numeric fields were also tested below, within and above their permitted ranges.

Grade entry tests utilized manual inputs structured around the supported eight-column module layout. Failure conditions included a missing student identifier, unsupported data types, incomplete row submission, and grades that could not be normalized safely. ML evaluation used a held-out test portion of the synthetic dataset; the same held-out observations were not used to fit the regression coefficients.

5.3.2 Acceptance Basis

A functional test was considered successful when the system produced the required state or output and did not expose an unhandled error. For invalid input, success meant rejecting the request and returning an understandable message. For integration tests, success additionally required that the receiving component interpreted identifiers, fields and response states consistently. Prediction availability was accepted only when the lifecycle router selected an applicable model; returning an explanatory unavailable message was the correct outcome when neither previous GPA nor current CA evidence existed.

5.4 Functional Testing Results

The principal student, administrator and server-side workflows operated as required after the implementation corrections recorded in Chapter 4. The functional results below summarize representative tests rather than claiming an exhaustive enterprise regression suite. Each result is tied to visible application behavior or a structured server/database response.

ID	Test condition	Observed result	Status
F01	Valid active student credentials	Student dashboard opened and session created	Pass
F02	Incorrect password or unknown StudentID	Access rejected with a readable error	Pass
F03	Inactive provisioned student	Access rejected before dashboard loading	Pass
F04	Duplicate StudentID in administration	Duplicate record rejected	Pass
F05	Valid programme, batch, semester and module data	Records saved and available to dependent views	Pass
F06	Complete and valid manual grade input	Grades accepted and normalized	Pass
F07	Incomplete grade submission	Submission rejected, prompting student to complete entry	Pass
F08	Missing module or invalid numeric input	Validation error returned; no false grades stored	Pass
F09	Ordinary grades with configured credits	Semester GPA calculated using credit weighting	Pass
F10	Multiple completed semester records	CGPA calculated from total weighted points and credits	Pass
F11	AB, MC or NE result value	Special state flagged instead of treated as a numeric grade	Pass

Table 5.3: Functional results - access, administration and result processing

ID	Test condition	Observed result	Status
F12	Previous GPA available; current CA unavailable	Lifecycle router selected Model A	Pass
F13	Previous GPA unavailable; current CA available	Lifecycle router selected Model B	Pass
F14	Previous GPA and current CA available	Lifecycle router selected Model C	Pass
F15	Neither previous GPA nor current CA available	Prediction withheld with an explanatory message	Pass
F16	Missing or out-of-range prediction field	Request rejected before inference	Pass
F17	Attendance below configured requirement	Attendance warning returned independently of GPA estimate	Pass
F18	Insufficient study hours for configured workload	SLQF-derived workload warning returned	Pass

F19	Applicable CA condition not satisfied	Condition-specific CA warning returned	Pass
F20	Completed DASS-21 responses	Depression, anxiety and stress categories evaluated separately	Pass
F21	Calendar event created and retrieved	Student-owned event displayed in the calendar workflow	Pass
F22	Academic prompt submitted to chatbot	Advisory response or controlled provider error displayed	Pass

Table 5.4: Functional results - prediction, warnings and student support

5.4.1 Functional Findings

The lifecycle tests were particularly important because selecting the wrong model would create a technically valid response from inappropriate evidence. The corrected router consistently used Model A, B or C only when its required academic evidence existed. The no-evidence case correctly produced an unavailable message, which confirms that PrePal does not promise a meaningful prediction to a brand-new student before the minimum academic baseline becomes available.

Result-processing tests also confirmed the need for controlled failure. A complete and valid grade submission is processed directly, but an incomplete or improperly formatted manual entry is rejected instead of producing a misleading GPA. This behaviour is important because the final implementation explicitly relies on manual grade entry to guarantee 100% data integrity.

5.5 Integration Testing Results

Integration tests evaluated whether independently deployed components exchanged the correct academic context and failure states. The tests followed both student and administrator workflows. Particular attention was given to identifiers, lifecycle values and policy data because an incorrect mapping could affect several later outputs even when each component worked correctly in isolation.

Integration path	Verified result	Status
Android to Firestore	Student identity, active state, profile, batch and semester context retrieved for the signed-in user	Pass
React panel to Firestore	Validated academic structures, students, policies and activity data written and retrieved	Pass
Administrator data to student workflow	Updated semester/module/policy configuration became available to the relevant student path	Pass
Android to Flask prediction route	Validated lifecycle and questionnaire fields produced a structured prediction or unavailable response	Pass
Flask to model artifacts	The selected Model A, B or C artifact loaded on the server and returned an estimate	Pass
Android to validation route	Manual grade inputs, student identifier and validation responses were exchanged without embedding validation mapping logic in the app	Pass
Android to conversational-AI route	Prompt, advisory response and service-error state were returned to the chat interface	Pass
Failure across a cloud boundary	Network/provider failure produced a controlled message rather than an application crash or fabricated result	Pass

Table 5.5: Integration testing results

5.5.1 End-to-End Evaluation

The complete student path was exercised from account verification and Firestore context loading through result analysis, questionnaire generation, Flask processing, model routing, warning evaluation and Android result presentation. This test demonstrated that the visible student output depended on administrator-maintained programme data, current lifecycle evidence and server-side processing rather than on hard-coded values inside the interface. The administrative path was exercised by changing relevant PrePal records and confirming that the student workflow consumed the updated context. This validated the role of Firestore as the shared data layer. It did not demonstrate synchronization with the official Saegis Campus student-management system, because that connection was deliberately outside the implemented architecture.

5.5.2 Integration Limitations

The integration result is dependent on internet connectivity and the availability of Firestore, Render, Groq and Gemini. Testing confirmed controlled failure handling, but it did not remove this dependency. No offline model, local result-processing service or institutional data mirror was implemented. External provider quotas, Render cold starts and network quality can therefore influence the apparent speed and availability of otherwise correct functions.

5.6 Performance Evaluation

5.6.1 Machine-Learning Evaluation

Models A, B and C were trained and evaluated as separate Multiple Linear Regression estimators because each represents a different point in the academic lifecycle. The 1,000-record synthetic dataset was divided into training and held-out test portions. Model performance was evaluated using R-squared, Mean Absolute Error and Mean Squared Error. The same metric set was used for all three models to support a consistent comparison.

Metric	Purpose	Interpretation in PrePal
R-squared	Proportion of variation in synthetic Future_GPA explained by the model	Higher values indicate stronger fit within the evaluated dataset; they do not establish institutional validity.
Mean Absolute Error (MAE)	Average absolute distance between predicted and generated GPA	Read directly in GPA points and therefore useful for explaining typical prototype error.
Mean Squared Error (MSE)	Average squared prediction error	Penalizes larger errors more strongly and helps reveal occasional poor estimates.

Table 5.6: Metrics used for Tri-Model evaluation

Numerical reporting note: The exact final R-squared, MAE and MSE console values were not included in the evidence available while this chapter was prepared. They are therefore not reconstructed or estimated here. The submitted report should reproduce the exact values from the final training notebook or model-evaluation output so every figure is traceable to the same experimental run.

The available technical evidence confirms that all three models completed training, evaluation, artifact saving and Flask inference. Coefficient and correlation reviews were used during development to identify implausible relationships and data leakage, and the dataset-generation logic was revised to represent continuous assessment, attendance,

workload, sleep and stress-related patterns more defensibly. Nevertheless, the measured scores describe how well the regressions reproduced patterns deliberately created in synthetic data. They must not be interpreted as proof that the same error level will occur with real Saegis Campus students.

5.6.2 Service and Response Performance

Interactive performance was assessed qualitatively during functional and integration testing. Firestore reads and writes supported the required account, configuration and calendar workflows under development conditions. Standard Flask prediction requests required only validation, model selection and regression inference, while manual grade entry workflows required structural validation against configured module arrays. Processing manual grade input was inherently faster and computationally lighter than automated text extraction methods, which is another significant performance benefit of the manual entry constraint.

Response time was not measured through a formal benchmark suite, concurrent-user simulation or sustained load test. Render cold starts, mobile network quality, and external AI-provider response time can materially affect an individual request. Consequently, this chapter does not state an unsupported average latency, throughput or maximum-user figure. Production deployment would require measured percentile response times, concurrency tests, error-rate monitoring, and capacity planning.

5.6.3 Performance Validity

The technical performance results are sufficient to show that the architecture can execute the intended prototype workflows, but they are not sufficient for institutional deployment. A stronger evaluation would retrain and test the models with de-identified official historical data, keep a genuinely unseen validation cohort, compare the MLR models with appropriate baselines, examine error across programmes and student groups, and monitor prediction drift across semesters. These activities were beyond the completed project scope.

5.7 Usability Evaluation

Usability was evaluated formatively through implementation-stage task walkthroughs, repeated interface checks and supervisor feedback. The evaluation concentrated on whether a student or administrator could understand the next action, whether irrelevant inputs were hidden, whether errors were recoverable and whether advisory outputs were described accurately. It was not a formal large-sample user study and no System Usability Scale score or statistically representative satisfaction result is claimed.

Usability aspect	Observed implementation	Evaluation
Role clarity	Student and administrator access are separated between Android and web interfaces	Reduces exposure to irrelevant controls
Navigation	Back navigation and task-focused screens were retained or corrected during iteration	Supports return to the appropriate dashboard or previous step
Input relevance	Previous-GPA and CA fields are shown according to lifecycle availability	Avoids requesting evidence a student does not yet have
Validation feedback	Required, range, grade entry format and network errors are presented as readable messages	Supports correction instead of silent failure
Result comprehension	GPA estimate, reasons, academic warnings and DASS-21 messages are distinguished	Reduces confusion between statistical, rule-based and wellbeing output
Academic analytics	GPA/CGPA, weak modules and semester trend are presented through focused views	Supports retrospective interpretation without manual calculations
Administrative efficiency	Browser forms, lists and CSV functions organize configuration work	Better suited to structured administration than mobile controls
Recovery and availability	Cloud-service failures return an error or fallback state	Prevents an unavailable service from appearing as a valid result

Table 5.7: Formative usability evaluation

5.7.1 Improvements Resulting from Evaluation

- Administrative functions were moved to the React web panel so wide forms, record lists and CSV operations were not forced into the student mobile interface.
- Duplicate StudentID validation was added to prevent ambiguous student access and result ownership.
- CGPA presentation was corrected to use cumulative credit weighting rather than an unweighted average of semester GPAs.
- Questionnaire fields and model routing were revised so the interface asks only for evidence available at the student's current lifecycle stage.
- Prediction reasons and condition-specific warnings were added so an estimated GPA is accompanied by understandable factors requiring attention.

- DASS-21 output wording was separated from academic prediction and framed as awareness or referral guidance rather than diagnosis.
- Progress and failure messages were added around Firestore, Flask, manual grade validation and conversational-AI operations.

5.7.2 Usability Limitations

A large-scale usability evaluation with undergraduates from every degree programme was not completed. The prototype therefore has no defensible completion-time distribution, task-success percentage, satisfaction score or accessibility-conformance result for the intended campus population. Android-only availability also excludes iOS users from direct evaluation, while different devices, screen sizes, network conditions and programme-specific module structures may create grade entry usability issues not observed during development.

Before campus-wide deployment, representative students and administrators should perform structured user-acceptance tasks covering sign-in, manual grade entry, analytics, each prediction lifecycle, warning interpretation, DASS-21 wording, calendar use, chatbot guidance and academic administration. Results should record task completion, defects, response time, severity and participant feedback without treating wellbeing responses as clinical data. Any resulting interface or wording changes should then be regression-tested across the complete integrated workflow.

5.7.3 Overall Evaluation

Within the final-year project boundary, PrePal achieved its principal technical goal. The Android student interface, web administration panel, shared database, Tri-Model prediction service, manual grade entry processing, warning rules, wellbeing awareness and conversational support operate as one prototype. Functional and integration tests verified the principal paths and controlled error states.

The evidence is strongest for implementation correctness and weakest for real-world predictive validity, production performance and population-level usability. PrePal should therefore be presented as a technically evaluated academic-support prototype that still requires official data validation, security hardening and representative user evaluation before institutional deployment.

5.8 Evaluation of the Spaced-Repetition Hypothesis

The spaced repetition hypothesis proposes that distributing learning activities across multiple study sessions can support longer-term retention more effectively than concentrating the same learning into one session. Within PrePal, this hypothesis is relevant because regular study behaviour, study-hour management and early academic intervention form part of the system's academic-support rationale.

PrePal provides several functions that can indirectly encourage distributed learning. Students can manage academic events and reminders through the calendar, receive

warnings when reported study hours are insufficient relative to the configured workload, and obtain study-planning guidance through the AI chatbot. GPA predictions and weak-module information may also encourage students to begin corrective study activities before final examinations.

However, the implemented system does not contain a complete spaced-repetition engine. It does not:

- Create flashcards or recall questions automatically.
- Measure whether a student remembered a specific learning item.
- Calculate increasing review intervals.
- Reschedule learning material according to recall performance.
- Compare spaced learning with massed learning experimentally.
- Measure whether spaced repetition improved examination marks or GPA.

Therefore, the spaced repetition hypothesis cannot be accepted or rejected using the results of this project. PrePal demonstrates supporting mechanisms such as reminders, workload warnings and conversational study guidance, but it does not provide evidence that an adaptive spaced-repetition intervention caused an improvement in academic performance. The hypothesis should consequently be treated as a theoretically relevant future direction rather than an evaluated achievement of the implemented system. A future version could introduce module-level revision items, recall-quality ratings and an interval-scheduling algorithm. Its effectiveness could then be evaluated using study-plan adherence, retention-test results, module marks and changes in semester GPA.

5.9 Critical Evaluation of the System

PrePal successfully demonstrates the technical feasibility of integrating academic-result analysis, lifecycle-sensitive GPA prediction, academic-risk identification, wellbeing awareness, administration and conversational support within one prototype. Nevertheless, its results must be considered together with its technical and evaluation limitations.

5.9.1 Strengths of the System

A major strength is the Tri-Model prediction structure. Model A, Model B and Model C allow prediction to use the information available at different stages of the student lifecycle. This is more appropriate than requiring every student to provide the same academic evidence regardless of whether previous GPA or current continuous-assessment marks are available.

The system also maintains a clear distinction between different outputs. GPA and CGPA are calculated from grades and credits, future GPA is a statistical estimate, academic warnings are generated through configured rules, and DASS-21 results provide non-diagnostic wellbeing awareness. This separation reduces the risk of presenting every result as a machine-learning prediction.

Hosting the model artifacts and processing logic within the Flask service improves maintainability. Models and warning conditions can be revised on the server without

embedding updated model files in every Android installation. Separating the React administration panel from the Android student application also prevents administrative controls from overcrowding the student interface.

The implementation of a strictly validated manual grade entry system ensures maximum result-processing data integrity. Explicit handling of AB, MC and NE values within the entry interface also prevents exceptional academic states from being incorrectly converted into ordinary grade points.

5.9.2 Weaknesses and Risks

The most important limitation is the use of synthetic training data. Although the models were technically evaluated using R-squared, MAE and MSE, these results show how well the models reproduce relationships created in the synthetic dataset. They do not establish predictive accuracy for real Saegis Campus students.

Multiple Linear Regression also assumes approximately linear relationships between the predictors and semester GPA. Actual academic performance may be influenced by nonlinear relationships, interactions and unmeasured factors that the models cannot represent.

Prediction reliability is further affected by self-reported attendance, study hours, sleep duration, continuous-assessment information and DASS-21 responses. The system does not verify these values through biometric attendance, an LMS, wearable devices or an official institutional database.

Manual grade entry depends on the student accurately transcribing their grades based on their programme's module structure. Programme-specific result layouts and unexpected module arrangements may create confusion for students during the entry process. Because automated PDF extraction was removed to prioritize data integrity, the system lacks an automated upload alternative if a student finds manual transcription overly tedious.

The system is also dependent on Android devices, internet connectivity, and multiple cloud services. Firestore, Render, Groq, or Gemini failures may temporarily interrupt important functions. The custom Firestore authentication approach is suitable for a prototype but requires stronger password protection, institutional identity management and security testing before production deployment.

5.9.3 Overall Critical Assessment

PrePal should be evaluated as a successful integrated academic prototype rather than a production-ready institutional system. It achieves its principal technical objectives and demonstrates a coherent architecture for predictive and advisory student support. However, official data validation, enhanced entry automation, security hardening, formal performance testing and representative user evaluation are necessary before campus-wide operational use.

5.10 Evaluation of Methodology and Tools

5.10.1 Evaluation of the Development Methodology

An Agile and iterative development approach was appropriate because the system contained several independent but connected technologies. Android, React, Firestore, Flask, machine-learning and external-AI functions could be implemented incrementally and integrated after their individual workflows became stable.

The methodology also allowed supervisor feedback to influence the final design. Important changes included moving administration to a web panel, correcting CGPA credit weighting, preventing duplicate StudentID values, implementing lifecycle-dependent model routing, improving prediction explanations, refining the manual grade validation logic and adopting DASS-21 awareness processing.

However, the project did not apply every formal Scrum practice. There is no claim of a full Scrum team, formally measured sprint velocity or production product backlog. Testing and documentation were also constrained by the final-year project schedule. The methodology is therefore more accurately described as practical Agile iteration supported by repeated feedback and technical correction.

5.10.2 Evaluation of the Selected Tools

The native Android Java/XML stack was suitable for implementing the student-facing mobile application and supported device-level navigation, document selection, local sessions and academic visualizations. Its principal limitation is that the final student application is Android-only.

React, Vite and TypeScript were appropriate for the administration panel. They supported reusable browser-based forms, structured record management and CSV operations. TypeScript also helped define predictable administrative data structures.

Cloud Firestore provided a practical shared database for the Android and React clients. Its managed cloud service and flexible document structure supported rapid development. However, its NoSQL design does not automatically enforce relational integrity, meaning programme, batch, semester, module and student references must be validated by application logic.

Python, Flask and scikit-learn were suitable for data processing and Multiple Linear Regression development. They supported model training, evaluation, artifact storage and REST-based deployment. Flask also allowed prediction and warning logic to remain outside the Android package. Render simplified deployment but introduced possible cold-start delays and online dependency.

The decision to implement rigorous manual grade entry instead of automated document extraction provided a highly reliable result-processing solution. Validated form inputs process directly and instantly. Nevertheless, manual entry speed remains sensitive to the student's familiarity with the interface and the complexity of their specific programme module structure.

Groq with Gemini fallback increased the availability of conversational academic support. However, generated responses may be incomplete or inaccurate, and both services introduce availability, privacy and provider-dependency concerns. Their output must remain advisory.

5.10.3 Overall Methodology and Tool Assessment

The selected methodology and tools were effective for producing an integrated final-year prototype within the available time and resource constraints. The technologies were assigned distinct responsibilities and supported iterative correction. The main weaknesses arise not from basic tool suitability, but from synthetic training data, external-service dependence, prototype authentication and the limited scale of formal evaluation.

6. Future Work

6.1 Introduction

The completed PrePal system demonstrates an integrated prototype for academic-result analysis, lifecycle-sensitive GPA prediction, academic-risk warnings, DASS-21 awareness, calendar support, administration and conversational guidance. Future work should concentrate on converting this prototype evidence into institutionally validated, secure and maintainable operation. The improvements described in this chapter are not presented as completed functionality.

Development priorities arise directly from the limitations identified during implementation and evaluation. The most important work is real-data validation and governance, followed by production security, programme-wide configuration, representative user evaluation and measured service performance. Additional learning-support features should be introduced only after the reliability of the existing academic and account workflows has been established.

Future-work principle: Enhancements should strengthen evidence, safety and institutional reliability before adding highly visible features. A larger feature set would not compensate for an unvalidated model, weak identity controls or inaccurate academic data.

6.2 Institutional Data and Model Validation

The highest-priority future task is to obtain appropriate institutional approval and a de-identified historical dataset containing the academic and behavioural variables needed by the prediction models. Data collection would require privacy review, clearly defined retention rules, restricted access and documentation of how each field was produced. Missing values, inconsistent programme rules and changes across academic years would need to be identified before model development.

Models A, B and C should then be retrained and evaluated using real records while preserving the lifecycle boundaries implemented in the prototype. A final evaluation should

keep a genuinely unseen cohort or later semester for validation, report R-squared, MAE and MSE, and compare performance across programmes, batches and academic stages. Results should also be compared with simple baselines, such as predicting the training-set mean or using previous GPA alone, so that the additional predictors demonstrate measurable value.

6.3 Prediction, Explainability and Monitoring

Future research could compare Multiple Linear Regression with regularized regression and carefully selected nonlinear methods. A more complex model should be adopted only if it produces repeatable improvement on real unseen data and remains explainable to students and academic staff. Prediction ranges should remain within the institutional GPA scale and should be accompanied by clear limitations.

- Add formal prediction intervals or another calibrated confidence indicator so students can see uncertainty rather than a single apparently exact number.
- Evaluate model error across degree programmes, batches and relevant student groups to identify systematic performance differences.
- Introduce versioning for datasets, model artifacts, features and academic policies so each returned estimate can be traced to the correct configuration.
- Monitor prediction drift and error after each semester and define conditions for retraining, review or model withdrawal.
- Improve explanation logic using validated feature contributions while avoiding causal claims from observational relationships.

6.4 Academic Data and Result-Processing Integration

A production version should integrate with an approved Saegis Campus student-management or Learning Management System interface if the institution makes one available. Such integration could reduce duplicate administration and self-reported academic data, but it would require formal authorization, data contracts, access controls and reconciliation rules. The current PrePal administration panel should remain the source only for information that is intentionally managed inside PrePal.

Manual grade entry currently ensures absolute data integrity but places the transcription burden on the student. A future enhancement should explore integrating a highly robust Optical Character Recognition (OCR) engine or a digital PDF parser to automate this process. However, any automated extraction feature must require explicit student review and validation before data is saved, ensuring that the 100% data integrity established by the prototype's manual entry constraint is not compromised by parsing errors.

6.5 Personalized Learning and Spaced Repetition

A future learning module could convert weak-module information, examination dates and available study time into a structured revision plan. This should be implemented as a transparent scheduling function rather than being represented as an existing chatbot capability. Students should be able to review, edit and accept suggested activities instead of having an external AI service silently control the plan.

Adaptive spaced repetition could be introduced through module-level learning items, recall-quality ratings and interval scheduling. This could be further enhanced by automatically generating interactive quizzes to test student comprehension before major examinations. The system would need to record when an item was reviewed, whether it was recalled and when it should appear again. Its effectiveness should be evaluated through retention tests, adherence and module outcomes rather than assumed from reminder use alone. A past-paper repository could also be considered only with institutional permission and appropriate copyright controls.

6.6 Platform, Accessibility, Localization and Offline Support

The student service could be extended beyond Android through an iOS application or responsive web interface. Shared interface specifications and API contracts would help each platform present the same lifecycle, warning and advisory boundaries. Accessibility evaluation should include screen-reader labels, text scaling, colour contrast, touch-target size, keyboard operation for the administration panel and alternatives to colour-only weak-module indicators.

Furthermore, future iterations should incorporate multi-language implementations (localization) to support a diverse student body, ensuring the interface, academic warnings, and wellbeing guidance are accessible in students' preferred languages.

Limited offline support could cache non-sensitive profile, calendar and previously viewed analytics data. Locally created events might be queued for synchronization after connectivity returns. Offline prediction would require a separate security and model-update decision because distributing model artifacts to devices would conflict with the current server-managed approach.

6.7 Security, Privacy and Production Readiness

Production deployment requires replacing or strengthening the prototype account workflow. Future identity management could use an institutionally governed authentication service, secure password hashing designed for credentials, multi-factor authentication for administrators, password recovery controls, session expiry and auditable authorization. Firestore rules should be independently reviewed and tested against cross-user and cross-role access attempts.

- Perform threat modelling, dependency review, penetration testing and secure configuration review for Android, web, Firestore and Flask components.

- Move secrets and provider keys into controlled deployment configuration and rotate them according to an operational policy.
- Define consent, privacy notices, retention periods, deletion procedures and access logging for academic and wellbeing information.
- Encrypt sensitive information in transit and apply additional protection at rest where institutional risk assessment requires it.
- Establish backup, recovery, monitoring, incident response and service-availability procedures before campus-wide use.

6.8 AI and Wellbeing Governance

Conversational support could be grounded in approved programme documents, examination by-laws and student-support information through a controlled retrieval process. Responses should identify their advisory status and direct students to official sources for decisions. Provider performance, privacy terms, prompt handling and inappropriate responses would require ongoing review.

DASS-21 messages should be reviewed with qualified wellbeing professionals and connected to accurate, current referral pathways. The system should continue to avoid diagnosis and emergency decision-making. Clear escalation wording, consent and limits on sensitive-data retention are more important than increasing the number of wellbeing predictions.

6.9 Evaluation and Deployment Roadmap

Stage	Principal work	Intended evidence
Critical foundation	Institutional approval, real-data validation, identity/security redesign and privacy governance	Evidence and controls required before production use
Pilot readiness	Programme configuration, expanded manual entry format validations, monitoring, backups and measured performance	Stable operation for a limited authorized pilot
User validation	Structured UAT with representative students and administrators plus accessibility review	Verified task completion, usability findings and correction backlog
Controlled pilot	Limited deployment with support, incident reporting and model-error monitoring	Operational evidence without claiming full campus readiness
Longitudinal evaluation	Compare predictions and support use with later official outcomes across semesters	Assessment of real predictive validity and potential academic value

Extended learning support	Spaced repetition, interactive quizzes, editable study plans and permitted learning resources	Added only after the core academic service is dependable
Store Distribution	Deploy the Android application to the Google Play Store	Real-world use, controlled updates and seamless installation for the campus population

Table 6.1: Prioritized future-development roadmap

This ordering prevents optional learning features from delaying the work needed to establish validity, security and institutional trust. Each stage should have explicit acceptance criteria and should be reviewed before the next expansion is authorized.

6.10 Future-Work Summary

The most valuable next step is not simply to add more functions. It is to validate the existing predictive core with appropriate real data, secure the system for institutional use and demonstrate usability with representative users. Once those foundations are established, PrePal can be extended carefully through broader platform access, official data integration, model uncertainty, adaptive revision support and governed AI assistance.

7. Conclusions

7.1 Introduction

This chapter concludes the PrePal final-year project by relating the completed system to the original academic problem, project aim, implemented objectives and evaluation evidence. It summarizes the contribution of the prototype without extending the claims beyond the functions and testing described in the report.

7.2 Response to the Identified Problem

The project addressed the absence of an integrated student-facing mechanism for early academic self-assessment, lifecycle-sensitive GPA estimation, structured academic-risk feedback and general academic support. Existing academic records primarily describe completed performance, whereas PrePal was designed to help an undergraduate interpret available evidence before the end of a current semester.

PrePal responds to this problem through one connected prototype. It calculates GPA and CGPA from supported results, highlights weak modules and performance trends, selects an applicable prediction model according to available evidence, evaluates configured academic conditions, provides DASS-21 awareness messages, and offers calendar and conversational support. Administrators maintain the academic structures and policies required by these workflows through a separate web panel.

7.3 Achievement of the Aim and Objectives

The principal project aim was successfully achieved. The system transitioned from a conceptual design to a fully implemented, multi-component prototype that addresses the stated problem. The specific achievements are mapped directly against the original project objectives defined in the proposal:

- I. For carrying out an analysis on past academic performance records of undergraduate students: *Achieved*. The project successfully analyzed historical academic structures and utilized a 1,000-record dataset reflecting Saegis Campus regulations to validate the prediction engine.
- II. To design and implement machine learning models capable of predicting future examination outcomes based on past grades: *Achieved*. The Tri-Model Multiple Linear Regression suite (Models A, B, and C) was successfully implemented to dynamically predict semester GPA based on the student's available past grades and continuous assessments.
- III. To develop a mobile application that allows students to upload their academic results in PDF format or manually enter their academic results: *Partially Achieved (Constraint Applied)*. A robust native Android application was successfully developed. However, as documented throughout this report, the PDF upload functionality was explicitly discarded during the design phase. The system instead

relies exclusively on a strict manual grade entry interface to guarantee 100% data integrity and prevent automated transcription errors.

- IV. To work on performance analytics to identify weak modules, GPAs, and indicators of academic risk: *Achieved*. The Android application successfully processes student inputs to calculate GPA/CGPA, identify weak modules, and evaluate configured academic conditions to deliver proactive risk warnings.
- V. To implement the AI chatbot function that offers relevant academic support informed by performance data: *Achieved*. An AI chatbot, powered by the Groq API (with Gemini fallback), was integrated into the mobile app to provide on-demand, contextual academic guidance.
- VI. To generate personalized study plans and academic recommendations aligned with predicted outcomes: *Achieved*. The chatbot provides tailored study recommendations based on the student's queries and academic context, while structured automated scheduling is identified as a future enhancement.
- VII. In order to incorporate the academic calendar functionality to help plan for future examinations and academic events: *Achieved*. A personal event calendar with local notification support was successfully implemented in the student application.
- VIII. Aspects relating to the effectiveness and usability of the proposed system for independent undergraduate use: *Achieved*. The effectiveness of the Tri-Model architecture was validated technically, and the interface underwent formative usability checks to ensure role clarity and independent navigation for students.

7.4 Main Project Contributions

7.4.1 Lifecycle-Sensitive Prediction

The principal technical contribution is the lifecycle-sensitive routing of three regression models. The system does not assume that every student has previous university results and current continuous-assessment marks at the same time. Model A, Model B and Model C therefore represent different evidence states, while a fourth outcome explains that prediction is not yet available.

7.4.2 Integrated but Distinct Forms of Support

PrePal combines deterministic academic calculations, statistical prediction, rule-based warnings, non-diagnostic wellbeing awareness and conversational guidance without treating them as equivalent. This distinction strengthens the academic honesty of the interface and supports more understandable output.

7.4.3 Maintainable Distributed Architecture

The architecture separates the Android student experience, React administration, Firestore data, Flask domain processing and external AI services. Model artifacts remain on the

server rather than inside the Android package, and administrative controls are not distributed through the student interface. This separation allows individual components to be revised while preserving clear responsibility boundaries.

7.5 Evaluation Outcome

Functional testing confirmed the principal account, administration, result-processing, GPA/CGPA, lifecycle-routing, warning, DASS-21, calendar and chatbot behaviours. Integration testing confirmed the major Android-Firestore, React-Firestore, Android-Flask, Flask-model and conversational-AI paths together with controlled failure states.

The model suite was technically evaluated using a held-out portion of a 1,000-record synthetic dataset and the selected R-squared, MAE and MSE measures. These results demonstrate that the training and inference pipeline operates, but they do not establish official prediction accuracy for real Saegis Campus students. Similarly, formative interface evaluation does not replace a representative usability or longitudinal effectiveness study.

7.6 Limitations of the Conclusion

The conclusion is bounded by synthetic model data, self-reported questionnaire values, manual grade transcription accuracy, Android-only student access, internet dependence, external provider availability and prototype authentication. PrePal is not directly integrated with the official Saegis Campus student-management system or LMS. It does not provide a prediction-confidence measure, adaptive spaced repetition, a standalone automated study-plan generator or a past-paper repository.

The DASS-21 component cannot diagnose a mental-health condition, and the AI chatbot cannot replace lecturers, academic advisers or counsellors. No production-scale load test, penetration test, campus-wide user study or longitudinal academic-outcome evaluation was completed.

7.7 Final Conclusion

PrePal demonstrates that academic analytics, lifecycle-sensitive Multiple Linear Regression, configurable warning rules, wellbeing awareness and conversational support can be integrated into one coherent student-centred prototype. The project transformed an initially broad academic-support idea into an implemented architecture with clear role separation, model-selection logic and documented failure boundaries.

The project should therefore be considered successful within its final-year prototype scope. Its value lies in the completed integration and the explicit recognition of where evidence is still required. With official data validation, stronger security, programme-wide configuration and representative evaluation, the architecture could provide a foundation for a more dependable institutional academic-support service.

Concluding statement: PrePal achieves its principal technical purpose as an implemented and evaluated prototype, while responsibly avoiding claims of production readiness, clinical authority or official academic accuracy.

8. Reflection

The PrePal project allowed me to apply knowledge from mobile development, web development, cloud databases, machine learning and API integration to a practical academic problem. At the beginning of the project, I mainly viewed PrePal as a GPA-prediction application. However, during development, I learned that a dependable prediction function requires much more than a trained model. It also requires accurate academic structures, correct GPA calculations, appropriate input validation, lifecycle-dependent model selection, clear warning messages and reliable communication between several system components.

The project changed considerably through supervisor feedback, implementation difficulties and testing. Some ideas from the original proposal were revised, integrated differently or removed from the final scope. These changes helped transform the initial concept into an implemented prototype with clearer technical and functional boundaries. This chapter reflects on the assumptions, design decisions, problem-solving methods and personal development that influenced the completed PrePal system.

8.1 Reflection on Initial Assumptions

At the beginning of the project, I assumed that developing the machine-learning model would be the most difficult part. During implementation, I discovered that training a Multiple Linear Regression model was only one part of a much larger system. The quality of the complete solution also depended on model-input availability, academic calculations, database configuration, result-document formats, network reliability and appropriate presentation of predictive information.

I initially assumed that one prediction model could support every undergraduate. However, students do not always have the same academic information. A new student may not have a previous university GPA, while another student may have previous results but not yet have current continuous-assessment marks. This led to the development of the Tri-Model architecture. Model A, Model B and Model C support different academic lifecycle conditions, while an unavailable pathway prevents prediction when the minimum information is not present.

I also assumed that a single questionnaire could be shown to every student. During implementation, I learned that requesting unavailable information could confuse students and produce unreliable inputs. The final design therefore determines the student's academic context and requests only the information required by the applicable model.

Another initial assumption was that GPA and CGPA calculation would be straightforward. Testing showed that calculating CGPA as a simple average of semester GPAs could produce an incorrect result when semesters contained different credit totals. I corrected the calculation so that cumulative grade points are divided by the total applicable credits.

I initially assumed that students would demand automated PDF result extraction. However, early architectural evaluations highlighted the immense risk of OCR transcription errors

corrupting the GPA predictions. I learned that in academic contexts, prioritizing 100% data integrity through validated manual entry is often a vastly superior choice to providing a flawed automated convenience.

I also assumed that deploying services in the cloud would make them continuously available. Development and testing demonstrated that network failures, Render cold starts, provider limits and external-service outages can interrupt prediction and chatbot functions. This encouraged me to treat loading states, failure messages and fallback behaviour as functional requirements.

Finally, I learned to distinguish between the system's intended audience and its demonstrated readiness. PrePal is designed for undergraduates across Saegis Campus through configurable programmes, batches, semesters and modules. However, complete programme-wide readiness cannot be claimed until the policies, modules and result formats of each programme have been configured and evaluated.

8.2 Reflection on Design Decisions

Selecting Multiple Linear Regression provided an understandable foundation for the prediction component. It allowed me to inspect coefficients and evaluate the models using R-squared, Mean Absolute Error and Mean Squared Error. Its interpretability was valuable for explaining the project. Nevertheless, academic performance may involve nonlinear and complex relationships that MLR cannot fully represent. I therefore learned that using an understandable algorithm does not remove the need for validation using real institutional data.

The Tri-Model architecture was one of the most important design decisions. It allowed the prediction process to reflect the information available at different academic stages. Testing the combinations of previous-GPA and current-CA availability demonstrated how a small routing error could select an inappropriate model. This taught me that the validation and model-selection logic surrounding a machine-learning model can be as important as the model itself.

Separating the student and administrator interfaces was another valuable decision. The Android application focuses on undergraduate activities, while the React web panel supports administrative forms, record management, CSV operations and policy configuration. A browser-based interface is more suitable for administrative work involving larger screens and structured data entry. I would retain this separation in future development.

Keeping the trained model artifacts on the Flask server was also appropriate. Models can be updated without embedding new artifacts in every Android installation. This improves maintainability and prevents model-management responsibilities from being placed in the student interface. The disadvantage is that prediction requires internet access and depends on the availability of the Render-hosted service.

Firestore accelerated development by allowing the Android application and React administration panel to use shared structured data. Its flexible document model was useful

for a prototype, but it also required careful application-level validation. The custom authentication process was sufficient for demonstrating the project, although I would replace it with institutionally governed identity management and stronger credential-specific password protection before production deployment.

The strict manual grade entry workflow guaranteed data integrity and eliminated the risk of automated transcription errors. However, relying entirely on manual result entry left students with a potentially tedious manual task if they have extensive academic histories. A future version should explore highly robust automated extraction that clearly identifies student-entered information, but only if absolute accuracy can be maintained.

Separating GPA prediction, academic warnings and DASS-21 output was also essential. A predicted GPA is a statistical estimate, academic warnings are produced from configured rules, and DASS-21 provides non-diagnostic wellbeing screening. Presenting these outputs with clear boundaries helped prevent the application from appearing to provide official academic decisions or clinical diagnoses.

The conversational AI component was implemented as an external support route using Groq with a Gemini fallback. This reduced the requirement to build a language model locally, but it introduced network, provider and response-accuracy dependencies. The chatbot must therefore remain a general advisory function and should not replace lecturers, counsellors or official institutional guidance.

8.3 Reflection on Problem Solving Approach

The practical Agile approach used during the project supported incremental development and revision. Functions were developed in smaller stages, demonstrated, tested and improved based on supervisor feedback. This was particularly useful because several important design weaknesses only became visible after the components were connected. My most effective debugging method was to separate an end-to-end problem into individual component boundaries. For example, when prediction failed, I checked the Android input, student lifecycle context, Firestore records, Flask validation, model-feature order and returned response separately before testing the complete workflow again. This was more reliable than modifying several components simultaneously.

Several problems contributed directly to my learning:

- Duplicate StudentID records showed that validation should occur before a database write rather than attempting to correct ambiguous records afterwards.
- The CGPA problem required me to return to the academic formula and test examples containing different semester-credit totals.
- Model-selection problems were addressed by testing every previous-GPA and current-CA availability combination, including the condition where prediction must remain unavailable.
- Unrealistic model behavior required examination of synthetic-data relationships, correlations, feature leakage, coefficients and edge cases instead of relying only on a high R-squared value.

- Manual entry validation edge cases were separated into grade normalization, credit weighting, exceptional state handling (AB, MC), and controlled rejection.
- External-service problems encouraged me to include progress indicators, understandable error messages and chatbot fallback behavior.

One area that I would improve is the systematic recording of test evidence. Maintaining a structured test repository containing exact inputs, expected outputs, screenshots, model-run results and defect histories from the beginning would have made the final evaluation easier to reproduce. I would also organize representative undergraduate testing earlier instead of leaving broad usability evaluation until the end of the project.

8.4 Reflection on Personal Growth

PrePal significantly expanded my technical knowledge because I had to connect mobile, web, cloud, machine-learning and external-service technologies. I gained practical experience in Android Java/XML development, React and TypeScript, Firestore document modelling, Flask API development, strict form validation logic, Multiple Linear Regression evaluation and cloud-deployment troubleshooting.

My understanding of machine learning became more critical. I learned that a high evaluation score alone does not prove that a model is useful, particularly when it has been trained using synthetic data. Dataset assumptions, feature leakage, train-test separation, coefficient interpretation, model routing and external validity must also be examined. This has helped me understand the difference between technical model evaluation and institutional validation.

The project also increased my awareness of security, privacy and ethical responsibility. Academic results, credentials and wellbeing responses are sensitive information. Password hashing, HTTPS and role checks alone are not sufficient to claim production security. Similarly, DASS-21 results and AI-generated responses require careful wording, privacy controls and clear referral boundaries.

I improved my communication and project-management skills by translating supervisor feedback into technical changes, explaining formulas and model inputs, documenting differences from the original proposal and keeping the Android, web and Flask components aligned. The repeated revisions taught me to treat feedback as evidence for improving the design rather than as a final-stage correction.

Most importantly, I learned that successful software engineering includes understanding what should not be claimed. PrePal is a meaningful and integrated final-year prototype, but its synthetic dataset, reliance on manual transcription, online dependencies, prototype authentication and limited user evaluation must remain clearly documented. Balancing technical achievement with honest limitations is one of the most valuable lessons I will carry into future academic and professional projects.

References

Dass21 questionnaire.

Akçapınar, G., Altun, A. and Aşkar, P. (2019) “Using learning analytics to develop early-warning system for at-risk students,” *International Journal of Educational Technology in Higher Education*, 16(1). Available at: <https://doi.org/10.1186/s41239-019-0172-z>.

Antony, M.M., Bieling, P.J., Cox, B.J., Enns, M.W. and Swinson, R.P. (1998) *Psychometric Properties of the 42-Item and 21-Item Versions of the Depression Anxiety Stress Scales in Clinical Groups and a Community Sample, Psychological Assessment*.

Credé, M., Roch, S.G. and Kieszczynka, U.M. (2010) “Class attendance in college: A meta-analytic review of the relationship of class attendance with grades and student characteristics,” *Review of Educational Research*. SAGE Publications Inc., pp. 272–295. Available at: <https://doi.org/10.3102/0034654310362998>.

Dawson, Shane. (2012) *Proceedings of the 2nd International Conference on Learning Analytics and Knowledge*. ACM.

Maheshwari, G. and Shaukat, F. (2019) “Impact of Poor Sleep Quality on the Academic Performance of Medical Students,” *Cureus* [Preprint]. Available at: <https://doi.org/10.7759/cureus.4357>.

Nonis, S.A. and Hudson, G.I. (2010) “Performance of College Students: Impact of Study Time and Study Habits,” *Journal of Education for Business*, 85(4), pp. 229–238. Available at: <https://doi.org/10.1080/08832320903449550>.

Shahiri, A.M., Husain, W. and Rashid, N.A. (2015) “A Review on Predicting Student’s Performance Using Data Mining Techniques,” *Procedia Computer Science*. Elsevier B.V., pp. 414–422. Available at: <https://doi.org/10.1016/j.procs.2015.12.157>.

Sri Lanka Qualifications Framework (SLQF) (no date).

Taylor, D.J., Vathauer, K.E., Bramoweth, A.D., Ruggero, C. and Roane, B. (2013) “The Role of Sleep in Predicting College Academic Performance: Is it a Unique Predictor?,” *Behavioral Sleep Medicine*, 11(3), pp. 159–172. Available at: <https://doi.org/10.1080/15402002.2011.602776>.

Wollny, S., Schneider, J., Di Mitri, D., Weidlich, J., Rittberger, M. and Drachsler, H. (2021) “Are We There Yet? - A Systematic Literature Review on Chatbots in Education,” *Frontiers in Artificial Intelligence*. Frontiers Media S.A. Available at: <https://doi.org/10.3389/frai.2021.654924>.

Yağcı, M. (2022) “Educational data mining: prediction of students’ academic performance using machine learning algorithms,” *Smart Learning Environments*, 9(1). Available at: <https://doi.org/10.1186/s40561-022-00192-z>.

Zhang, T., Zhong, Z., Mao, W., Zhang, Z. and Li, Z. (2024) “A New Machine-Learning-Driven Grade-Point Average Prediction Approach for College Students Incorporating Psychological Evaluations in the Post-COVID-19 Era,” *Electronics (Switzerland)*, 13(10). Available at: <https://doi.org/10.3390/electronics13101928>.

Zimmerman, B.J. (2002) “Becoming a self-regulated learner: An overview,” *Theory into Practice*. Ohio State University Press, pp. 64–70. Available at: https://doi.org/10.1207/s15430421tip4102_2.

Appendix A

These documents can be accessed via the following Google Drive repository: <https://drive.google.com/drive/u/0/folders/10Lhs3xTrriuAygKRZCqqJxogw8csRaIM>

Appendix B

The experimental code, machine learning models, and data processing logic developed for the Prepal application were created and tested using Google Colaboratory.

The interactive notebook containing the full source code, execution outputs, and visualizations has been hosted digitally. It can be viewed at the following link:

<https://colab.research.google.com/drive/1TZpKZL7W76nbVU0mcmMToKiLJzAsE3Os?usp=sharing>